

SAPIENZA
UNIVERSITÀ DI ROMA

Gait-ViViT: A Video Processing Model for Parkinson's Disease Detection

Faculty of Information Engineering, Informatics, and Statistics
Applied Computer Science and Artificial Intelligence

Gianmaria Romano
ID number 2105539

Advisor
Prof. Marco Raoul Marini

Academic Year 2025/2026

Gait-ViViT: A Video Processing Model for Parkinson's Disease Detection
Bachelor's Internship Report. Sapienza University of Rome

© 2026 Gianmaria Romano. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: romano.2105539@studenti.uniroma1.it

"When I was little I wanted to become God. The Christian God with a capital 'G.' When I was about five I realized this would never happen. So I accepted the fact that I would be a martyr."

— *Amélie Nothomb, Fear and Trembling*

Abstract

Parkinson's disease is a neurodegenerative disease that alters a patient's movements, with symptoms showing up as hyperkinetic or hypokinetic features. Based on existing research into the applications of machine learning frameworks to analysing Parkinson's disease, this project aims to develop a deep learning model for Parkinson's disease detection based on the subject's overall gait.

The chosen strategy consists of adapting the existing *Video Vision Transformer* backbone to a supervised anomaly detection task and training the model on frame sequences extracted from synthetic videos. The model has been trained using a hybrid transfer learning approach and, to achieve more robust results, K-fold cross-validation has been employed during training.

The results show that, using a small and imbalanced dataset, a transformer-based architecture tends to struggle with generalization, being especially vulnerable to false positive predictions, although a simpler few-shot action detection model is able to achieve more consistent performance for the same task even as the classification threshold varies.

Acknowledgements

First and foremost, I would like to thank the academic figures that guided me through this internship, giving me the possibility to experience a glimpse of the world of academic research. In particular, I would like to thank Professor Marini for the trust he placed in me and in this project by allowing me to complete my internship under the supervision of VisionLab, as well as Dr. Diko for his guidance and for pushing me to think outside the box.

Next, I would like to thank my parents, Giuseppe and Laura, for their constant support, especially during my internship. Putting up with me was not always the easiest thing in this world and I am eternally grateful for their presence and, most importantly, their patience.

Last, but definitely not least, I would like to thank all the friends I have made during this adventure for making this adventure human and light-hearted. A special thank you goes to Hayat and Samin, who have been by my side through thick and thin and with whom I shared countless wonderful moments. I would also like to thank Claudia and Zosia for all the experiences and projects we shared, especially in the study room, and for the memorable moments that came out of those. I am grateful to Alessandro, Alessio and Elnaz for the lightness and spirit they brought into my life during our friendship and for the fact that we were often on the same wavelength, as well as Dana, an amazing source of advice, both academic and hobby-related, and an icon of style like no one before.

Contents

Acknowledgements vii

1 Introduction 1
- 1.1 Aims and scope of the project . . . 1
- 1.2 Current state of the art . . . 2
- 1.3 Contribution and report outline . . . 3

2 Datasets 5
- 2.1 The Kaggle MMU Visual-Based Parkinson's Disease Dataset . . . 5
- 2.2 The F-BioGate Dataset . . . 6

3 Computer vision and video processing 7
- 3.1 Vision Transformer . . . 9
- 3.2 Swin Transformer . . . 10
- 3.3 Video Vision Transformer . . . 11
  - 3.3.1 Spatio-temporal attention . . . 12
  - 3.3.2 Factorised encoder . . . 12
  - 3.3.3 Factorised self-attention . . . 13
  - 3.3.4 Factorised dot-product attention . . . 13
- 3.4 TimeSformer . . . 14

4 Data preprocessing 17
- 4.1 Video rendering . . . 17
  - 4.1.1 Rendering the Kaggle MMU Visual-Based Parkinson's Disease Dataset . . . 17
  - 4.1.2 Rendering the F-BioGate Dataset . . . 18
- 4.2 Dataframe aggregation . . . 19
- 4.3 Frame extraction . . . 20
  - 4.3.1 Defining the frame extraction interval . . . 20
  - 4.3.2 Filtering out problematic frames . . . 21
  - 4.3.3 Cleaning and resizing frames . . . 22
  - 4.3.4 Storing frames . . . 23
- 4.4 Implementing a custom `Dataset`-like class . . . 24

5 Model implementation 25
- 5.1 Extracting tubelet embeddings . . . 28
- 5.2 Implementing a single encoder block . . . 28

5.2.1 Implementing the attention mechanism 28
5.2.2 Implementing the multilayer perceptron 29
5.3 Adapting the output layer 29
5.4 Initializing the model with pre-trained weights 30

6 Training **31**
6.1 Transfer learning 31
6.2 Constructing the training, validation and testing datasets 31
6.3 Learning configurations 32
6.3.1 Gradient optimization 32
6.3.2 Learning rate scheduler 33
6.3.3 Loss function 33
6.4 Implementing the training algorithm 34
6.4.1 Running linear probing on the classification head 34
6.4.2 Fine-tuning the entire model 34
6.4.3 Testing the best model 35

7 Results **37**
7.1 Performance evaluation metrics 37
7.2 Performance analysis 38
7.2.1 Testing alternative models 41
7.3 Threshold ablation study 41
7.3.1 Extending the ablation study to the few-shot model 42

8 Conclusions **43**
8.1 Strengths and current limitations 43
8.2 Future work 44

Chapter 1

Introduction

1.1 Aims and scope of the project

Parkinson's disease is a neurodegenerative disease of the central nervous system that affects both motor and non-motor systems.

This disorder is characterised by a set of motor symptoms, collectively known as "parkinsonism", that include both hyperkinetic features, such as rigidity or tremors, and hypokinetic features, such as bradykinesia or akinesia.

These symptoms are normally assessed using Part 3 of the Movement Disorder Society's Unified Parkinson's Disease Rating Scale (MDS-UPDRS), which contains 33 metrics aimed at evaluating the severity of a patient's motor symptoms [1].

While the MDS-UPDRS currently acts as the gold standard for assessing Parkinson's disease symptoms, it is necessary to address its key limitations.

In fact, the MDS-UPDRS tends to be less effective for analysing early or mild stages of the disease, during which the motor symptoms might be subtle or less relevant, and its scores are often characterised by high variability in scoring consistency, making evaluations more subjective based on each clinician's assessment practices.

Concerning the latter point, Sibley et al.'s review [2] on (remote) video-based assessments proposes points regarding an emerging approach that could be used for more objective assessments in the near future.

For instance, video-based analysis can help make the procedure more comfortable for patients, who, by avoiding the need to travel to a clinic for each assessment, can reduce health risks caused by off-medication states or external diseases.

Additionally, automated video assessments based on machine learning techniques can support clinicians in better identifying and diagnosing Parkinson's disease, providing a potential solution to the main limitations of the MDS-UPDRS.

However, video-based assessments still come with challenges and drawbacks that need to be addressed in order to achieve reliable results.

Firstly, the pipeline needs to take into consideration both ethical aspects, requiring informed consent from users to store their data while ensuring compliance with data protection regulations, and accessibility issues, requiring the implementation of an interface that can be easily used both by patients and clinicians.

In addition, since many computer vision algorithms are based on pose extraction and estimation, model performance will heavily depend on video consistency and,

more importantly, how well the input videos are able to capture Parkinson-related movements, although recent studies show that this hurdle can be overcome by integrating wearable or wireless sensors in video assessment.

Furthermore, some motor symptoms cannot be properly assessed using videos, requiring in-person evaluations instead, which can limit the usability and applications of video-based analyses.

Overall, despite the challenges and limitations it poses, computer vision-based analysis of Parkinson's disease appears to be a highly promising research field, as shown by the results achieved by existing models.

1.2 Current state of the art

The main challenge of automated assessments of Parkinson's disease lies in the lack of reliable biomarkers that can be used as a reference.

However, Dennis and Strafella's review article [3] suggests that machine learning techniques can overcome this obstacle by employing proper performance metrics, such as sensitivity, which denotes the number of correctly classified positive cases, and specificity, which denotes the number of correctly classified negative cases. Since the choice of adequate sensitivity and specificity values often depends on the sample size, it is common practice to assess model performance using the area under the receiving operating curve, which can be found by iteratively changing the threshold value of the classification variable and observing how sensitivity and specificity vary as a result.

Today, various machine learning and deep learning techniques, each with its own characteristics and limitations, are being used and combined to analyse Parkinson's disease symptoms and stages or to differentiate between Parkinson's disease and other atypical parkinsonisms.

For instance, *linear regression* allows to plot a linear relationship between two (or more) continuous variables, whereas *logistic regression* is used for discrete classification based on probabilities estimated by the sigmoid function

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{1.1}$$

Alternatively, a *decision tree* is a supervised learning model that, given a dataset, identifies an input variable that can be used to split data points into output classes, leading to another split with respect to a second input variable or to a complete prediction. This kind of model also includes *random forests*, which operate and merge multiple decision trees through a majority vote, and *gradient-boosting decision trees*, which gradually gather results and fix the model according to a loss function.

Similarly, *support vector machines* act as a supervised learning model that, based on a set of data points, aims to find a non-linear plane that best separates the data, typically in two different groups, although multi-class discrimination can be implemented by combining different support vector machines.

Lastly, *(artificial) neural networks* act as feed-forward models in which information flows through several layers that process data in order to produce some output. *Convolutional neural networks* are a particular type of neural network that is used

to process images by applying convolutional filters to extract a feature map whose information allows the model to classify the input image.

In the context of video-based assessment of Parkinson's disease, a state-of-the-art research is found in Guarín et al.'s video-based framework for (early) detection of Parkinson's disease from recordings of the finger-tapping task [4], using both feature-based classification models and time-series-based classification algorithms. All these models were trained using a multicenter dataset containing videos of both healthy control subjects and patients showing mild symptoms of bradykinesia, allowing the models to account for real-world subject variability as well. Additionally, the training procedure included subject-aware learning and cross-validation techniques to ensure better reliability of performance analysis.

The results show that both classification paradigms are able to achieve good discrimination results, although feature-based classifiers achieved consistently better scores compared to time-series-based classifiers. Further analysis revealed that feature-based classifiers mainly focused on kinematic features, such as opening and closing speed or amplitude decay during movements, that correspond to known motor dysfunctions exhibited by patients afflicted with Parkinson's disease.

Overall, this study confirms the importance of movement consistency in (early) detection of Parkinson's disease, potentially offering a support tool for detecting subtle anomalies that a clinician may not be able to identify by eye alone.

1.3 Contribution and report outline

Patients afflicted with Parkinson's disease are known to exhibit "Parkinsonian gait", a movement disorder that causes patients to struggle with movements as their stride length and walking speed are significantly reduced.

For this reason, this report aims to outline how existing video processing models can be adapted to detect whether a subject is affected by Parkinson's disease or not by detecting eventual anomalies in the way they move their limbs while walking.

In particular, the report will be structured as follows:

- **Chapter 2** will introduce the datasets used to train and test the model, analysing their contents and eventual limitations.
- **Chapter 3** will provide a general summary of how computer vision and video processing work, along with a brief description of some popular models.
- **Chapter 4** will explain the data acquisition and preprocessing procedure.
- **Chapter 5** will describe the architecture of the chosen framework and how this has been adapted for the task.
- **Chapter 6** will provide details on the training and testing procedures.
- **Chapter 7** will summarise the results obtained by the model.
- **Chapter 8** will address the conclusions that can be drawn from the model's overall performance.

Chapter 2

Datasets

The model will be trained using data from Connie et al.'s **Kaggle MMU Visual-Based Parkinson's Disease Dataset**, which is available on the Kaggle platform, and Musso et al.'s **F-BioGate Dataset** from Sapienza University of Rome. These datasets have been chosen because both contain data from videos of people walking, making them useful for a gait analysis-based detection of Parkinson's disease.

In particular, the important information about each video is stored in a dataframe, which contains the source of each video, which denotes whether it comes from the **Kaggle MMU Dataset** or the **F-BioGate Dataset**, and their file path, as well as information about each video's resolution and a binary label indicating whether the subject is healthy or has Parkinson's disease.

2.1 The Kaggle MMU Visual-Based Parkinson's Disease Dataset

Connie et al.'s **Kaggle MMU Visual-Based Parkinson's Disease Dataset** [5] contains data from **292 videos**, which come from sources like YouTube or Bilibili, of people walking.

In particular, this dataset provides **150 videos** of healthy subjects and **142 videos** of subjects afflicted with Parkinson's disease, with *Table 2.1* providing more details on the structure of the dataset.

Folder	Files	Description
`NORMAL`	150	Healthy subjects
`MILD`	29	Subjects showing mild symptoms of Parkinson's disease
`MODERATE`	61	Subjects showing moderate symptoms of Parkinson's disease
`SEVERE`	62	Subjects showing severe symptoms of Parkinson's disease

Table 2.1. A summary of the contents of the **Kaggle MMU Dataset**.

The main limitation of this dataset lies in the fact that it does not provide the actual videos used in the original research, likely due to privacy compliance reasons. Instead, the dataset provides JSON files containing the keypoints extracted from the video, including both spatial coordinates and an extraction confidence score.

For this reason, using this dataset first requires using the provided JSON files to render synthetic videos that will be used to train the model.

2.2 The F-BioGate Dataset

Musso et al.'s **F-BioGate Dataset** [6] contains **2448 videos** of subjects walking and moving up or down a flight of stairs. Along with standard RGB videos, this dataset also includes video depth maps, which capture z-axis distances, and infrared videos, which are able to better capture light.

However, using these `.avi` videos as they are would introduce several problems as the presence of background elements would cause a bias in the model during learning, making it unreliable. Therefore, after converting the files to `.mp4` format, a step that was carried out to avoid problems during keypoint extraction and rendering, each file is rendered in a similar way to the procedure used for the other dataset.

Additionally, since all subjects in this dataset are healthy, using all videos would introduce a strong class imbalance, thus requiring the introduction of resampling and weighting techniques to make model performance reliable.

Chapter 3

Computer vision and video processing

Computer vision is a subfield of artificial intelligence that deals with automating the tasks performed by the human visual system, such as extracting and processing information from digital images or videos, using machine learning and deep learning techniques.

Many applications of computer vision revolve around digital image processing, a research field born in the 1960s with the initial purpose of improving image quality.

Today, the most common image processing technique is (linear) filtering, which consists of performing a two-dimensional convolution between an image f and a filter kernel ω in order to obtain a filtered image g that can be described according to *Equation 3.1*.

$$g_{x,y} = w * f_{x,y} = \sum_{i=-a}^{a} \sum_{j=-b}^{b} w_{i,j} f_{x-i,y-j} \tag{3.1}$$

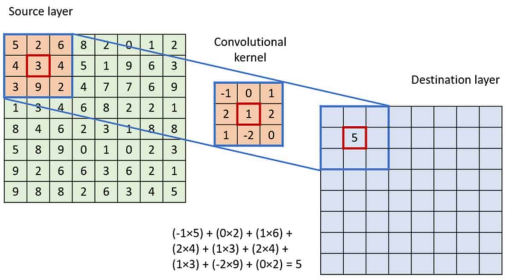


Figure 3.1. Example of a convolution operation.

For a long time, the state-of-the-art model for image processing has been represented by convolutional neural networks [7], which operate by implementing convolutional layers that convolve a small kernel across various patches of the input image in order to extract a feature map of relevant features that will be used to produce an output.

A key limitation of convolutional neural networks lies in the fact that, since training this kind of model on images tends to be too expensive, neurons in convolu-

tional layers are only connected to a smaller region of the input, which is known as their "receptive field". While this choice allows to reduce computational complexity during training, it significantly limits the way a convolutional neural network can extract information from an image as the model is restricted by local receptive fields and might fail to capture long-term pixel dependencies.

For this reason, the recent development of the (self-)attention mechanism, which was introduced for natural language processing tasks [8] as a more efficient alternative to encoder-decoder models, has led researchers to adapt transformers for computer vision tasks as these models can allow for a more comprehensive image processing pipeline.

In fact, current research is also focusing on implementing hybrid architectures that integrate both convolutional neural networks and self-attention using more computationally efficient techniques.

For instance, Pan et al. [9] have proposed a hybrid model that decomposes convolutional layers and self-attention into a two-stage procedure based on simpler 1×1 convolution operations.

In fact, this model first uses three 1×1 convolutions to project input features, which are reshaped into N pieces to obtain $3 \times N$ feature maps that are later used in parallel by two parallel branches. One of these branches implements multi-head self-attention by gathering the feature maps into N groups, each containing three pieces that will act as queries, keys and values, whereas the other branch uses a fully connected layer that, using a kernel of size k, generates k^2 feature maps that will be processed through shifting and aggregation.

Lastly, the outputs from the parallel branches are combined in order to convey as much information as possible:

$$F_{out} = \alpha F_{att} + \beta F_{conv}, \text{ where } \alpha \text{ and } \beta \text{ are learnable scalars.} \tag{3.2}$$

Overall, this proposed architecture allows to achieve great results on common image processing tasks while significantly reducing the memory overhead caused by the combination of convolutions and self-attention.

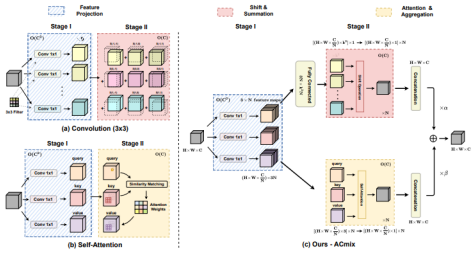


Figure 3.2. Sketch of Pan et al.'s *ACmix* model.

In this setting, a video can be seen as a sequence of frames, which contain spatial information for each instant, that are linked together by a temporal relationship.

For this reason, most video processing algorithms adapt computer vision techniques by introducing an additional axis representing time, which is crucial to grasp context and temporal information from the single frames, although this approach tends to be more expensive in terms of storage and computational complexity.

3.1 Vision Transformer

Based on the results obtained by Vaswani et al.'s *Transformer* architecture for natural language processing tasks, Dosovitskiy et al.'s *Vision Transformer* model [10] introduces an image classification architecture that is purely based on the attention mechanism, without implementing convolutional networks.

Since attention is commonly used to process a sequence of tokens, this model first reshapes a 2D image $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$, which has resolution (H, W) and features C channels, into a sequence of 2D patches $\mathbf{x}_p \in \mathbb{R}^{N \times P^2 C}$, where (P, P) denotes the resolution of each patch and $N = \frac{HW}{P^2}$ is the resulting number of patches, and adds learnable 1D position embeddings to each patch in order to retain positional information.

Next, a learnable embedding $\mathbf{x}_{class} = \mathbf{z}_0^0$ is prepended to the patch sequence, with its state at the end of the encoder, denoted as $\mathbf{z}_L^0$, representing the image representation $\mathbf{y}$. This embedding is ultimately passed to a classification head, which is implemented using a multi-layer perceptron with one hidden layer during pre-training and as a linear layer during fine-tuning.

The model's encoder architecture consists of alternating layers of multi-head self-attention and multi-layer perceptron blocks, and applies layer normalization before each block and introduces residual connections after each block.

Therefore, the encoder's behaviour can be summarised using the following set of equations:

$$\mathbf{z}_0 = [\mathbf{x}_{class}, \mathbf{x}_p^1\mathbf{E}; \mathbf{x}_p^2\mathbf{E}; \ldots; \mathbf{x}_p^N\mathbf{E}] + \mathbf{E}_{pos}, \text{ with } \mathbf{E} \in \mathbb{R}^{P^2C \times D} \text{ and } \mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}. \tag{3.3}$$

$$\mathbf{z}'_l = \mathrm{MSA}(\mathrm{LN}(\mathbf{z}_{l-1})) + \mathbf{z}_{l-1}, \text{ for each } l \in \{1, \ldots, L\}. \tag{3.4}$$

$$\mathbf{z}_l = \mathrm{MLP}(\mathrm{LN}(\mathbf{z}'_l)) + \mathbf{z}'_l, \text{ for each } l \in \{1, \ldots, L\}. \tag{3.5}$$

$$\mathbf{y} = \mathrm{LN}(\mathbf{z}_L^0) \tag{3.6}$$

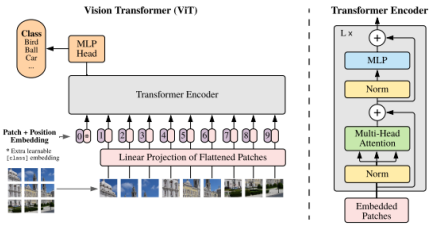


Figure 3.3. Overview of Dosovitskiy et al.'s *Vision Transformer* model.

Pre-training on common large datasets, such as ImageNet and JFT, reveals that the model is able to match, or even outperform, prior state-of-the-art convolution-based models, such as Kolesnikov et al.'s *Big Transfer* or Xie et al.'s *Noisy Student*, while also reducing computational costs.

However, training on smaller subsets of these datasets reveals that the model tends to perform worse than other models, such as *ResNet50*. This result can be justified by the fact that attention-based architectures lack the inductive bias

of convolutional networks and struggle to generalize on smaller datasets without regularization.

3.2 Swin Transformer

In order to mitigate the complexity of Dosovitskiy et al.'s *Vision Transformer* architecture, Liu et al. [11] introduce the *Swin Transformer* as a possible general-purpose backbone for transformer-based computer vision models.

In particular, this model operates using the "shifted windows" strategy, which consists of constructing a hierarchical feature map starting from smaller non-overlapping patches that are bridged across subsequent layers through simple window shifts.

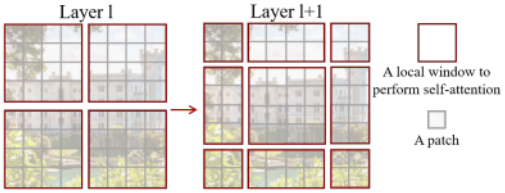


Figure 3.4. An illustration of the "shifted window" approach.

From a broader perspective, the model splits an input RGB image into non-overlapping patches, each treated as a token whose features consist of concatenations of raw pixel values, and, to produce a hierarchical representation of the input, the number of patches gets gradually reduced as the network gets deeper.

More specifically, patch tokens are processed using a Swin Transformer block, which adapts self-attention computations to exploit the shifted windows approach and limits computations to just local windows for better computational efficiency.

However, since a purely window-based mechanism has a limited modelling power due to the lack of connections across windows, the architecture implements a shifted window partitioning approach that connects neighbouring non-overlapping windows extracted in the previous layer.

In fact, the first block uses a regular window partitioning approach, whereas the following block adopts a shifted window approach based on the preceding layer, allowing to describe consecutive Swin Transformer blocks using the following set of equations:

$$\hat{\mathbf{z}}_l = \text{W-MSA}(\text{LN}(\mathbf{z}_{l-1})) + \mathbf{z}_{l-1} \tag{3.7}$$

$$\mathbf{z}_l = \text{MLP}(\text{LN}(\hat{\mathbf{z}}_l)) + \hat{\mathbf{z}}_l \tag{3.8}$$

$$\hat{\mathbf{z}}_{l+1} = \text{SW-MSA}(\text{LN}(\mathbf{z}_l)) + \mathbf{z}_l \tag{3.9}$$

$$\mathbf{z}_{l+1} = \text{MLP}(\text{LN}(\hat{\mathbf{z}}_{l+1})) + \hat{\mathbf{z}}_{l+1} \tag{3.10}$$

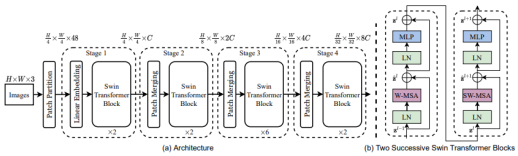


Figure 3.5. Overview of Liu et al.'s *Swin Transformer* model, with patch size 4×4.

Experiments conducted on common image processing tasks and datasets, such as ImageNet-1K for image classification or COCO for object detection, show that this model is able to outperform current state-of-the-art architectures, both using regular training and pre-training, providing a potentially effective and efficient approach to computer vision.

3.3 Video Vision Transformer

Since the attention mechanism is useful for modelling long-range dependencies within a sequence of tokens, Arnab et al.'s *Video Vision Transformer* [12] proposes a pure-transformer architecture as a more efficient model for video classification tasks compared to the prior state-of-the-art represented by 3D convolutional networks.

Since a video can be seen as a sequence of static images, known as "frames", this model first reshapes a video $\mathbf{V} \in \mathbb{R}^{T \times H \times W \times C}$, which consists of T frames with resolution (H, W) and C channels, into a sequence of tokens $\tilde{\mathbf{z}} \in \mathbb{R}^{n_t \times n_h \times n_w \times d}$, where n_t is the number of sampled frames and $n_h \cdot n_w$ is the number of patches. Then, the model adds the corresponding positional embeddings and reshapes the tokens into an input $\mathbf{z} \in \mathbb{R}^{N \times d}$ that is passed to the transformer.

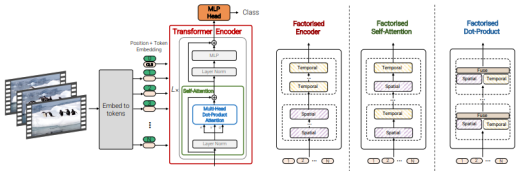


Figure 3.6. Overview of Arnab et al.'s *Video Vision Transformer* model and its variants.

A first method for extracting tokens from an input video consists of uniformly sampling n_t 2D frames, which are embedded using the same procedure of the *Vision Transformer* architecture, in order to extract $n_h \cdot n_w$ non-overlapping patches from each frame. These tokens are then concatenated together and forwarded to the transformer encoder block.

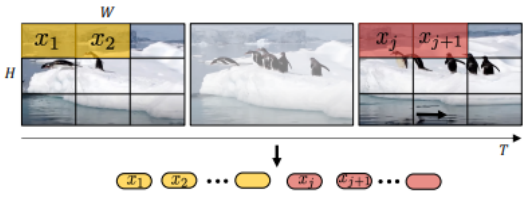


Figure 3.7. An illustration of the "uniform frame sampling" approach.

Alternatively, it is possible to extract non-overlapping spatio-temporal tubelets that are later linearly projected to $\mathbb{R}^d$, similarly to a 3D convolution. In particular, if tubelets of size $t \times h \times w$ are used, the model will extract a total of $n_t \cdot n_h \cdot n_w$ tokens, with $n_t = \lfloor \frac{T}{t} \rfloor$, $n_h = \lfloor \frac{H}{h} \rfloor$ and $n_w = \lfloor \frac{W}{w} \rfloor$, respectively.

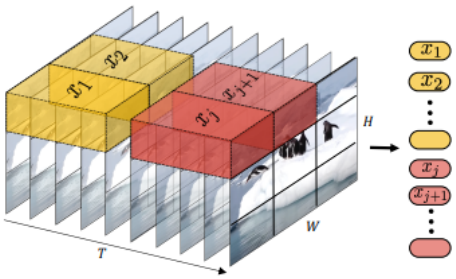


Figure 3.8. An illustration of the "tubelet embedding" approach.

3.3.1 Spatio-temporal attention

A first approach to video processing simply consists of extending the *Vision Transformer* architecture to model pairwise interactions between the extracted spatio-temporal tokens.

While this approach is very simple to implement, it is very inefficient as the quadratic complexity of the multi-head self-attention mechanism quickly explodes as more tokens are provided to the encoder.

3.3.2 Factorised encoder

The factorised encoder model makes use of two separate encoders.

Firstly, a spatial encoder models the interactions between tokens extracted from the same temporal index, creating representations $h_i \in \mathbb{R}^d$ for each temporal frame that are later concatenated into $\mathbf{H} \in \mathbb{R}^{n_t \times d}$, which is then forwarded to a temporal encoder to produce an output token for classification.

While this architecture features more encoder layers compared to the simple *Vision Transformer* extension, it is able to perform the same operations with a much lower computational complexity.

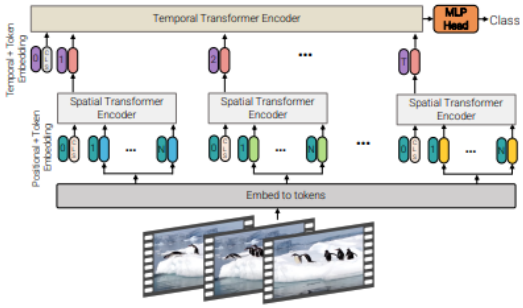


Figure 3.9. An overview of the *Factorised Encoder* model.

3.3.3 Factorised self-attention

The factorised self-attention model splits the self-attention mechanism into spatial self-attention, which models interactions between tokens from the same temporal index, and temporal self-attention, which models interactions between tokens from the same spatial index.

For this reason, the input tokens must be reshaped from $\mathbf{z} \in \mathbb{R}^{1 \times n_t \cdot n_h \cdot n_w \cdot d}$ to $\mathbf{z}_s \in \mathbb{R}^{n_t \times n_h \cdot n_w \cdot d}$, which is the sequence used to compute spatial self-attention, and $\mathbf{z}_t \in \mathbb{R}^{n_h \cdot n_w \times n_t \cdot d}$, which is the sequence used to compute temporal self-attention.

Therefore, the factorised self-attention model can be described using the following set of equations:

$$\mathbf{y}_s^l = \mathrm{MSA}(\mathrm{LN}(\mathbf{z}_s^l)) + \mathbf{z}_s^l \tag{3.11}$$

$$\mathbf{y}_t^l = \mathrm{MSA}(\mathrm{LN}(\mathbf{y}_s^l)) + \mathbf{y}_s^l \tag{3.12}$$

$$\mathbf{z}^{l+1} = \mathrm{MLP}(\mathrm{LN}(\mathbf{y}_t^l)) + \mathbf{y}_t^l \tag{3.13}$$

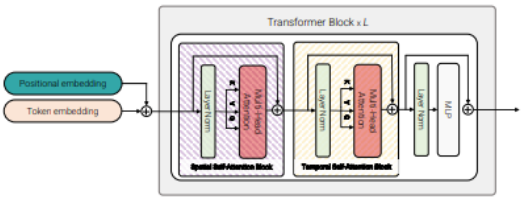


Figure 3.10. An overview of the *Factorised Self-Attention* variant of the model.

While this architecture introduces more parameters compared to the simple *Vision Transformer* extension, it is still able to achieve the same computational complexity as the factorised encoder architecture.

In addition, the order in which spatial and temporal self-attention scores does not influence model performance, provided model parameters are initialized using certain pre-training techniques, such as positional embeddings or embedding weights.

3.3.4 Factorised dot-product attention

A factorised dot-product attention model aims to maintain the same computational complexity as other factorised models while significantly reducing the number of parameters.

Generally speaking, performing the attention operation on an input $\mathbf{X}$ consists of performing a series of linear projections in order to find the queries $\mathbf{Q} = \mathbf{X}\mathbf{W}_q$, keys $\mathbf{K} = \mathbf{X}\mathbf{W}_k$ and values $\mathbf{V} = \mathbf{X}\mathbf{W}_v$ in order to compute the attention score

$$\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}, \text{ } d_k \text{ is the dimension of the key vectors.} \tag{3.14}$$

The idea of factorised dot-product attention is to construct $\mathbf{K}_s, \mathbf{V}_s \in \mathbb{R}^{n_h \cdot n_w \times d}$, which represent the keys and values used to compute attention scores $\mathbf{Y}_s$ for tokens

from the same spatial index, and $\mathbf{K}_t, \mathbf{V}_t \in \mathbb{R}^{n_t \times d}$, which represent the keys and values used to compute attention scores $\mathbf{Y}_t$ for tokens from the same temporal index. Lastly, the outputs of the various heads are concatenated and projected to find the final output.

Therefore, it is possible to describe the factorised dot-product attention mechanism using this set of equations:

$$\mathbf{Y}_s = \text{Attention}(\mathbf{Q}, \mathbf{K}_s, \mathbf{V}_s) \tag{3.15}$$

$$\mathbf{Y}_t = \text{Attention}(\mathbf{Q}, \mathbf{K}_t, \mathbf{V}_t) \tag{3.16}$$

$$\mathbf{Y} = \text{Concat}(\mathbf{Y}_s, \mathbf{Y}_s)\mathbf{W}_O \tag{3.17}$$

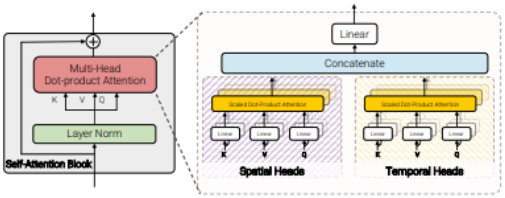


Figure 3.11. An overview of the *Factorised Dot-Product Attention* model.

3.4 TimeSformer

The limitations of convolutional neural networks in video processing, such as the restrictions posed by the inductive bias when modelling dependencies and training costs, have led Bertasius, Wang and Torresani to introduce *TimeSformer* [13], a purely self-attention-based video architecture.

To reduce the computational cost of the standard attention mechanism for video processing tasks, this architecture implements a "divided" attention mechanism that separately computes spatial attention and temporal attention.

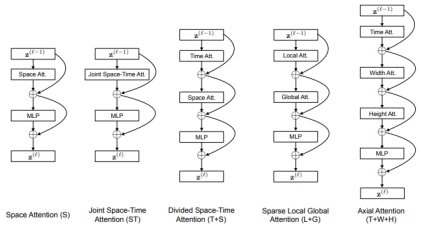


Figure 3.12. An illustration of various video self-attention blocks.

In fact, the model takes an input video $\mathbf{V} \in \mathbb{R}^{H \times W \times 3 \times F}$, which consists of F RGB frames of resolution (H, W), and decomposes each frame into N non-overlapping patches, each of size (P, P), that are flattened into simpler vectors $\mathbf{x}_{(p,t)} \in \mathbb{R}^{3P^2}$, where $p \in \{1, \ldots, N\}$ is a spatial index and $t \in \{1, \ldots, F\}$ is a temporal index.

Using a learnable matrix $\mathbf{E} \in \mathbb{R}^{D \times 3P^2}$ and a learnable positional embedding $\mathbf{e}^{pos}_{(p,t)} \in \mathbb{R}^D$, which encodes spatiotemporal positions, these vectors are then linearly

mapped into a sequence of embedding vectors

$$\mathbf{z}^0_{(p,t)} = \mathbf{E}\mathbf{x}_{(p,t)} + \mathbf{e}^{pos}_{(p,t)}, \text{ with } \mathbf{z}^0_{(p,t)} \in \mathbb{R}^D. \tag{3.18}$$

After prepending a learnable vector $\mathbf{z}^0_{(0,0)} \in \mathbb{R}^D$, which acts an embedding for the classification vector, this sequence is then passed to the transformer for attention computation.

To reduce computational costs, each encoder block in the transformer first computes temporal attention between patches located at the same spatial location and, using the resulting temporal encoding $\mathbf{z}^{l\ \text{time}}_{(p,t)}$, computes spatial attention between patches from the same frame so that the resulting spatial encoding $\mathbf{z}^{l\ \text{space}}_{(p,t)}$ can be used to compute the final encoding $\mathbf{z}^l_{(p,t)}$ of block l.

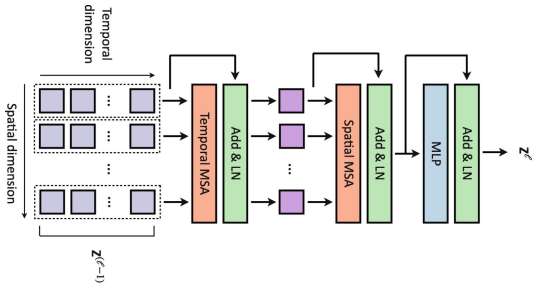


Figure 3.13. An overview of the *Divided Space Time Attention* mechanism.

Evaluating the architecture on common action recognition datasets, such as Kinetics-400 or Something-Something-V2 reveals that a pre-trained model is able to achieve state-of-the-art performance using the divided space-time attention mechanism, offering a scalable approach that is also less computationally demanding.

Other attention mechanisms tried on the model were "sparse local global" attention, which computes local attention scores for neighbouring patches followed by global attention scores within the video, and "axial" attention, which decomposes computations over time, width and height, although neither of these alternatives provided significant performance results compared to divided space-time attention.

Chapter 4

Data preprocessing

4.1 Video rendering

Since the starting datasets contain different kinds of files, the first preprocessing step consists of rendering each file into a synthetic `.mp4` video in order to avoid bias during training.

More specifically, each synthetic video consists of a black background and features an extracted skeleton, which is constructed using AlphaPose's *Halpe 26-Keypoint Pose Estimation and Tracking Model* [14], that is drawn using green circles to represent body parts and blue lines to represent joints and connections.

4.1.1 Rendering the Kaggle MMU Visual-Based Parkinson's Disease Dataset

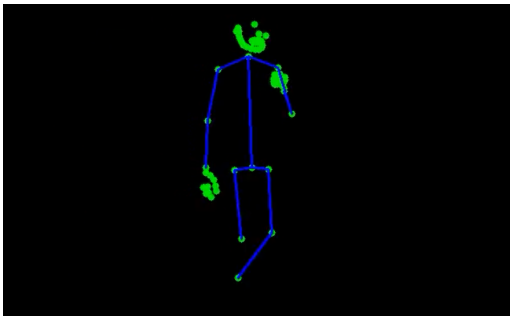
Since the **Kaggle MMU Visual-Based Parkinson's Disease Dataset** already provides JSON files containing the keypoints extracted from the original videos, the rendering function for this dataset simply focuses on creating the frames of each synthetic video, using **OpenCV**'s `VideoWriter` class to create a video object [15] and the **NumPy** library to create each frame [16].

More specifically, the rendering function is implemented to create synthetic videos with resolution 1280×720, which acts as a standard resolution for high-definition videos, and a frame rate of 30 frames per second.

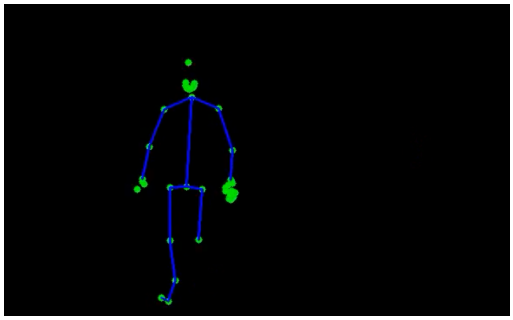
Regarding the frame creation algorithm, the first step is to create a black background using a $1280 \times 720 \times 3$ array, meaning that each frame can be seen as a RGB image of size 1280×720.

Next, the function iterates through the keypoints that were extracted for the current frame and, for each keypoint, it extracts the corresponding spatial coordinates and their confidence score, which is a value between 0 and 1 that denotes the probability with which the point was sampled from AlphaPose's pose estimation heatmap. However, the rendering function draws a keypoint on the frame if and only if its confidence score is greater than 0.2: this threshold is introduced in order to filter out noise in each frame.

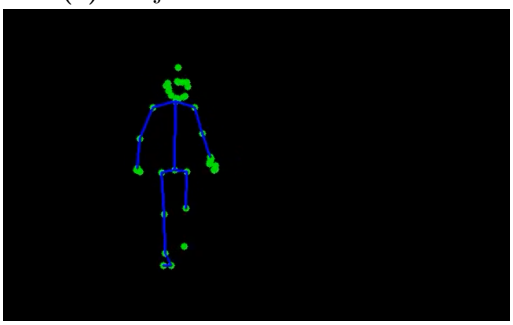
Lastly, the rendering function uses the 26-point skeleton model to draw the connections between the keypoints, provided both points do not fall out of range and both points have been saved on the frame.



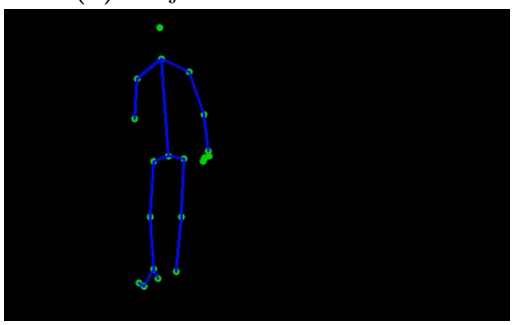
(a) Subject in the `NORMAL` folder.



(b) Subject in the `MILD` folder.



(c) Subject in the `MODERATE` folder.



(d) Subject in the `SEVERE` folder.

Figure 4.1. Rendered videos from the **Kaggle MMU Dataset**.

After all frames have been rendered, the function uses the `ffmpeg` command-line tool to make the video readable and saves it in a dedicated folder.

4.1.2 Rendering the F-BioGate Dataset

Since the **F-BioGate Dataset** provides actual `.avi` videos, the rendering procedure will require extracting keypoints from each video, which will be carried out using the AlphaPose model for consistency with the rendering done on other dataset, and using such keypoints to create the synthetic videos.

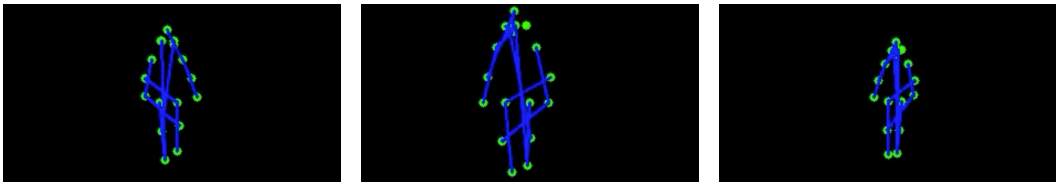
However, since AlphaPose models struggle with processing videos in the `.avi` format, each video in the dataset is converted into the `.mp4` format using the `ffmpeg` command-line tool.

Next, the AlphaPose repository, along with additional packages and model weights, is cloned and compiled in order to perform the keypoint extraction procedure. Since the repository also contains C++ and CUDA modules, this procedure is carried out using a GPU for compatibility purposes while also speeding up the extraction procedure.

After the model has been set up, its `demo_inference.py` script is used to extract the keypoints from each video, processing multiple videos in parallel to reduce the time needed to process the full dataset. In particular, since the script appeared to hang despite successfully extracting the keypoints from the videos, a timeout was implemented in order to free the GPU and collect the extracted keypoints.

Lastly, after the keypoints have been extracted from each video, a second rendering function generates the corresponding synthetic videos.

This function is implemented using a similar approach to the one described in *Section 4.1.1* for the **Kaggle MMU Visual-Based Parkinson's Disease Dataset**, albeit with some slight differences. In fact, this function integrates both **Imageio** [17] and **OpenCV** during video object creation in order to avoid compatibility issues and it manages keypoint extraction for each frame in a slightly different manner due to the structure of the JSON files created by the AlphaPose model. Additionally, since the AlphaPose model extracts the keypoints in parallel, this rendering function also needs to sort the extracted keypoints in order to ensure the synthetic video accurately represents the original movements. Furthermore, the height and width parameters of this rendering functions are changed to match the resolution of the original videos, which is 848×480 instead of 1280×720.



(a) Subject descending. **(b)** Subject ascending. **(c)** Subject walking.

Figure 4.2. Rendered RGB videos from the **F-BioGate Dataset**.

The results of the rendering procedure reveal that the AlphaPose model struggled with extracting keypoints from depth videos, although it appeared to work well when processing infrared and RGB videos.

For this reason, to ensure that videos from the two data sources can be merged without using potentially dirty data, only the synthetic videos generated from videos in the RGB format will be included in the complete dataset.

4.2 Dataframe aggregation

After all videos have been rendered, all the important information is stored in a dataframe.

This dataframe contains information about each video's origin, such as whether the video comes from the **Kaggle MMU Visual-Based Parkinson's Disease Dataset** or from the **F-BioGate Dataset** and the corresponding file path, as well as information about each video's resolution and the corresponding class label, which indicates whether the subject is healthy or is afflicted with Parkinson's disease.

Source Dataset	Videos
Kaggle	292
F-BioGate	816

(a) Videos by source dataset.

Class	Videos
`0` (Healthy)	966
`1` (Parkinson)	142

(b) Videos by class label.

Table 4.1. Some information about the video dataframe.

As indicated in *Table 4.1*, the dataframe contains information about **1108 videos**. In particular, *Table 4.1b* reveals that **only 142** of these videos represent a subject

with Parkinson's disease, creating a strong class imbalance.

For this reason, dedicated techniques, such as subsampling and anomaly detection, need to be introduced in order to make the training procedure more reliable.

4.3 Frame extraction

After completing the rendering procedure described in *Section 4.1* and storing the important information about each file in the dataframe mentioned in *Section 4.2*, the next step consists of extracting the frames that will be used to train the model.

However, since the original videos come from different datasets and represent subjects doing different actions, the synthetic videos might feature some noise hindering skeleton visibility, requiring to extract frames from specific intervals, during which the skeleton is more visible, and clean the sampled frames to remove, or at least reduce, noise.

In accordance to the techniques described in Arnab et al.'s paper [12, p. 3], the initial pipeline focused on uniformly sampling 32 frames from each synthetic video and denoising them, eventually discarding corrupted or low-quality frames, and resizing each frame to $(224, 224)$, which is the frame resolution used to train the original *Video Vision Transformer* model. Choosing to extract 32 frames per video stems from the goal of better capturing subtler movements shown by patients with Parkinson's disease.

While this approach is relatively simple, it failed to completely remove noise from the extracted frames, especially for videos coming from the **F-BioGate Dataset**, likely due to some bugs in the AlphaPose model used during the rendering procedure.

For this reason, the chosen pipeline swaps the two steps, first cleaning the video and then sampling the frames.

In fact, after loading a video, the frame extraction function first discards all the invalid or low-quality frames and then uniformly samples 32 frames among the remaining ones. Each of the sampled frames is then cleaned and resized to the target resolution of $(224, 224)$, eventually replacing any corrupted frames with black frames or duplicate frames as a fallback.

It is also worth noting that the number of frames to sample and the target size are chosen as standard parameters for compatibility with the *Video Vision Transformer* model, although the frame extraction function is implemented to work with different parameters as well.

4.3.1 Defining the frame extraction interval

The frame extraction interval denotes the time window during which the frames will be sampled from a video.

Since, as mentioned above, skeletons might be visible at different time intervals across different videos, the chosen frame extraction interval will vary depending on the input video.

Regarding videos coming from the **Kaggle MMU Visual-Based Parkinson's Disease Dataset**, the frames will be extracted throughout the entire video. This choice stems from the fact that these videos tend to be much shorter but still feature a clearly visible skeleton throughout the entire duration of the video.

By contrast, the frame extraction interval chosen for videos coming from the **F-BioGate Dataset** will depend on the action performed in the original video, as described in *Table 4.2*.

Action	Interval
Stairs down	From halfway through the video onwards
Stairs up	From the start of the video up to 75% of its duration
Walk	From halfway through the video up to 70% of its duration

Table 4.2. Frame extraction intervals for the **F-BioGate Dataset**.

This decision comes from the fact that, based on the action performed in each video, the skeleton may become less visible during certain intervals because the subject happens to be too close or too far from the camera.

4.3.2 Filtering out problematic frames

A key limitation of AlphaPose lies in the fact that the model can make mistakes during pose estimation, such as missing a person or creating multiple skeletons from redundant keypoints erroneously catching noise from the original video, a problem already addressed in other related studies, such as Gama et al.'s research on infant pose estimation from videos [18].

To avoid introducing a training error due to video noise, the frame extraction procedure makes use of confidence-based uniform frame sampling.

In fact, the frame extraction function first loads all the frames of the input video and, among those frames falling in the chosen extraction interval, checks which candidate frames are valid using a helper function.

The helper function takes as inputs a JSON file containing the keypoints extracted for the aforementioned video and a frame key, which is an index used to recover the keypoint information for a given frame, along with a confidence threshold value, which is set to 0.5 by default. More specifically, the function indicates to discard the frame if it does not exist in the keypoint file or if the corresponding keypoints are empty, which means that no body had been detected during the rendering procedure, or contain information about multiple skeletons. Additionally, the function discards all frames that, despite featuring a single skeleton, have an average keypoint confidence that falls below the chosen threshold.

After discarding the low-quality frames thanks to the helper function, the main function uniformly samples 32 frames among the remaining ones, using the `np.linspace` function to ensure that the frames are evenly distributed within the extraction interval but close to one another. It is also worth noting that, in the event that less than 32 valid frames remain after filtering, the `np.linspace` function repeats some indices, which means that some frames will be duplicated.

If the frame sampling was successful, the function starts processing each frame sequentially. Otherwise, it returns an array of 32 black frames of the desired size as a fallback value.

4.3.3 Cleaning and resizing frames

If the function manages to find and sample some valid frames from the video, the next step of the pipeline consists of cleaning and resizing the sampled frames in order to return a **NumPy** array of shape $(32, 224, 224, 3)$, where $(224, 224)$ is the default target resolution of the frames.

The first implementation of this procedure iteratively read and cleaned each sampled frame, eventually using a black frame as a fallback placeholder for the frames that **OpenCV** failed to read, and returned the array containing the processed frames.

While simple, this approach can be problematic during training as the model might be given black frames interrupting the action sequence, potentially causing issues during the learning procedure.

For this reason, the final implementation of the frame cleaning procedure introduces a buffer that replaces corrupted frames with a duplicate of the previous (valid) frame.

In fact, the function takes each frame and, if **OpenCV** was able to read it, it converts the frame from **OpenCV**'s BGR colour scheme to the standard RGB scheme, also applying a partial cropping to remove unfiltered noise when processing videos from the **F-BioGate Dataset**. Next, the function applies a `medianBlur` filter, which is useful to reduce noise while preserving edges, and resizes the cleaned frame to the target resolution of $(224, 224)$.

By contrast, if **OpenCV** was not able to read a frame, it will replace it with a duplicate of the last valid frame that was processed by the function. However, since this substitution does not work when the corrupted frames are at the start of the sequence, an additional loop will replace these frames with duplicates of the first valid frame in the sequence.

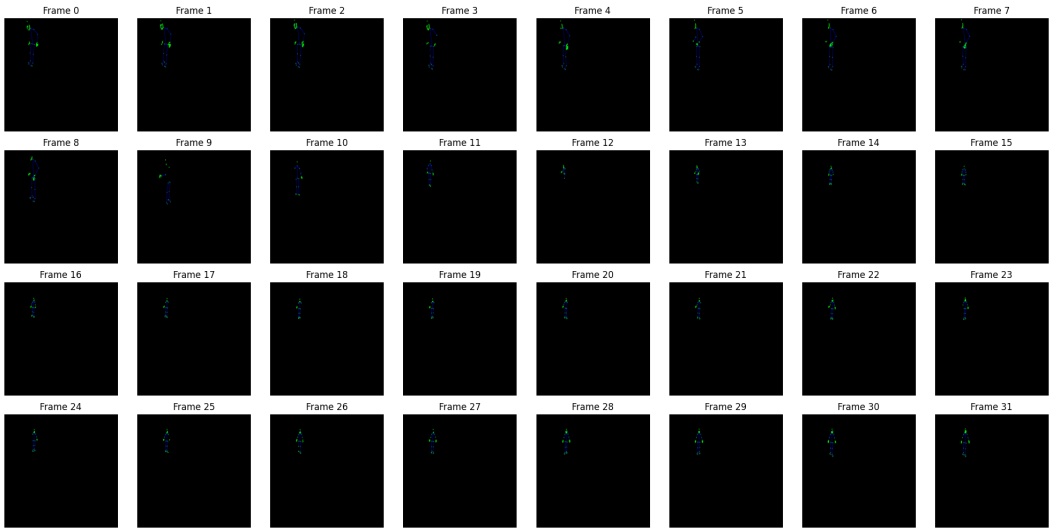


Figure 4.3. Frames extracted from a video in the **Kaggle MMU Dataset**.

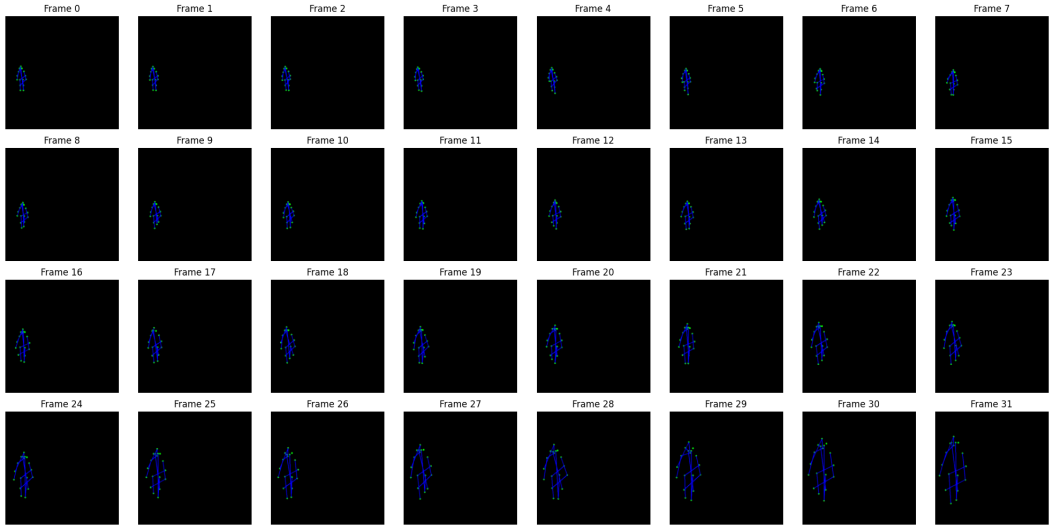


Figure 4.4. Frames extracted from a video in the **F-BioGate Dataset**.

At the end of this loop, the function will return an array containing the processed frames, eventually returning an array of black frames if the function was not able to read any frame.

4.3.4 Storing frames

The pipeline is ultimately applied to all the videos featured in the dataframe described in *Section* 4.2.

For each row in the dataset, the video and the corresponding keypoints file are retrieved in order to call the frame extraction function, asking to return 32 frames of size $(224, 224)$ for each video. The only differences in these function calls lie in the frame extraction interval, which is chosen according to the logic described in *Section* 4.3.1, and the confidence threshold: videos coming from the **Kaggle MMU Visual-Based Parkinson's Disease Dataset** use a confidence threshold of 0.15, which is chosen because, even though the skeletons tend to be visible throughout the videos, their keypoints have lower confidence scores, whereas videos coming from the **F-BioGate Dataset** use the default confidence threshold of 0.5.

The returned **NumPy** arrays are then converted into **PyTorch** tensors [19] for storage, provided the frame extraction procedure returned 32 frames.

Next, a new dataframe is created in order to keep track of the extracted frames for each video in the dataset, although only the files for which non-empty frames have been sampled will be included in this dataframe.

More specifically, each row of the new dataframe will contain the original video's identifier and source, along with the path to the tensor containing the extracted frames and the label associated to the video.

Source Dataset	Videos
Kaggle	248
F-BioGate	814

(a) Tensors by source dataset.

Class	Videos
`0` (Healthy)	947
`1` (Parkinson)	115

(b) Tensors by class label.

Table 4.3. Some information about the frame tensor dataframe.

Table 2.1 reveals that the new dataframe will contain information about **1062 videos**, meaning that **46 of the original videos** were discarded due to an unsuccessful frame extraction procedure.

In particular, *Table 4.3b* states that **only 115** of the saved videos represent a subject with Parkinson's disease, maintaining the class imbalance observed at the end of the rendering procedure.

4.4 Implementing a custom `Dataset`-like class

After extracting frames from the videos in the dataset, a custom `GaitViViTDataset` class, which extends **PyTorch**'s `Dataset` abstract class used to represent (known) datasets, is created in order to quickly extract data during training using a `DataLoader`.

Instances of this class are created using the `__init__` method, which loads the tensor dataframe and applies the indicated **Torchvision** transforms [20] to each tensor in the dataframe.

Additionally, this class also features the `__len__` method, which returns the number of elements found in the provided dataframe, and the `__getitem__` method, which reads a chosen row and loads the corresponding tensor, normalizing it [21] [22] and applying eventual transforms, and the label associated to the original video, returning these two elements as outputs.

Chapter 5

Model implementation

The model for the task will be implemented from scratch starting from **Hugging Face**'s `VivitForVideoClassification` class [23], which is closely based on the architecture described by Arnab et al. in their research.

```
VivitForVideoClassification(
  (vivit): VivitModel(
    (embeddings): VivitEmbeddings(
      (patch_embeddings): VivitTubeletEmbeddings(
        (projection): Conv3d(3, 768, kernel_size=(2, 16, 16), stride=(2, 16, 16))
      )
      (dropout): Dropout(p=0.0, inplace=False)
    )
    (layers): ModuleList(
      (0-11): 12 x VivitLayer(
        (attention): VivitAttention(
          (q_proj): Linear(in_features=768, out_features=768, bias=True)
          (k_proj): Linear(in_features=768, out_features=768, bias=True)
          (v_proj): Linear(in_features=768, out_features=768, bias=True)
          (o_proj): Linear(in_features=768, out_features=768, bias=True)
        )
        (layernorm_before): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
        (layernorm_after): LayerNorm((768,), eps=1e-06, elementwise_affine=True)
        (mlp): VivitMLP(
          (activation_fn): FastGELUActivation()
          (fc1): Linear(in_features=768, out_features=3072, bias=True)
          (fc2): Linear(in_features=3072, out_features=768, bias=True)
        )
        (dropout): Dropout(p=0.0, inplace=False)
      )
```

```
    )
    (layernorm): LayerNorm((768,), eps=1e-06,
    elementwise_affine=True)
  )
  (classifier): Linear(in_features=768, out_features=400, bias
   =True)
)
```

Listing 5.1. Structure of the `VivitForVideoClassification` class model.

Given a batch of B tensors of shape (C, T, H, W), the `GaitViViT` model first extracts non-overlapping tubelet embeddings, which contain both spatial and temporal information, and prepends a classification token before adding positional embeddings.

Next, the token sequence passes through the transformer encoder block, which consists of 12 blocks alternating between attention computations and multilayer perceptrons.

Lastly, layer normalization, which consists of normalizing features to stabilize gradient adjustment [24], is applied before passing the output representation of the classification token into the classification head, which returns a vector containing B class logit, one for each tensor in the input batch.

```
class GaitViViT(nn.Module):
  def __init__(self, img_size=(224, 224), patch_size=(16, 16),
    num_frames=32, tubelet_size=2, in_channels=3, embed_dim
   =768, depth=12, num_heads=12, mlp_ratio=4.0, qkv_bias=True,
    cls_ratio=2.0, dropout=0.0):
    super().__init__()

    # Extract the embeddings and determine the number of
   spatial and temporal batches.
    self.patch_embed = VivitEmbeddings(
        img_size=img_size,
        patch_size=patch_size,
        num_frames=num_frames,
        tubelet_size=tubelet_size,
        in_channels=in_channels,
        embed_dim=embed_dim,
        dropout=dropout
        )
    self.num_spatial_patches = self.patch_embed.emb.
   num_spatial_patches
    self.num_temporal_patches = self.patch_embed.emb.
   num_temporal_patches

    # Create and initialize the [CLS] token and the patch
   embeddings.
    self.cls_token = nn.Parameter(torch.zeros(1, 1, embed_dim)
   )
    self.pos_embed = nn.Parameter(torch.zeros(1, 1 + self.
   num_spatial_patches * self.num_temporal_patches, embed_dim)
   )
    nn.init.normal_(self.cls_token, std=0.02)
```

```
22      nn.init.normal_(self.pos_embed, std=0.02)
23
24      self.pos_drop = nn.Dropout(dropout)
25
26      # Create the transformer encoder block using the
        VivitLayer class.
27      self.blocks = nn.ModuleList([
28          VivitLayer(
29              num_spatial_patches=self.num_spatial_patches,
30              num_temporal_patches=self.num_temporal_patches,
31              embed_dim=embed_dim,
32              num_heads=num_heads,
33              mlp_ratio=mlp_ratio,
34              qkv_bias=qkv_bias,
35              dropout=dropout
36          )
37          for _ in range(depth)
38      ])
39
40      # Apply layer normalization before entering the
        classification head.
41      self.norm = nn.LayerNorm(embed_dim, eps=1e-6,
        elementwise_affine=True)
42
43      # Enter the classification head.
44      self.head_hf = int(embed_dim * cls_ratio)
45      self.head = AnomalyHead(in_features=embed_dim)
46
47  def forward(self, x):
48      # Determine the batch size.
49      B = x.shape[0] # Remember that x will have shape (B, C, T,
        H, W).
50
51      # Extract patch embeddings.
52      x = self.patch_embed(x)
53
54      # Prepend the classification token to the patch sequence
        and add the positional embeddings.
55      cls_tokens = self.cls_token.expand((B, -1, -1))
56      x = torch.cat([cls_tokens, x], dim=1)
57      x = x + self.pos_embed
58      x = self.pos_drop(x)
59
60      # Pass through the transformer encoder.
61      for block in self.blocks:
62          x = block(x, self.num_spatial_patches, self.
        num_temporal_patches)
63
64      x = self.norm(x)
65
66      # Extract the classification token that will be passed to
        the classification head.
67      cls_output = x[:, 0, :]
```

```
68      logit = self.head(cls_output)
69
70      return logit
```

Listing 5.2. Structure of the `GaitViViT` model.

5.1 Extracting tubelet embeddings

The `VivitEmbeddings` block deals with extracting spatio-temporal information from the input and later applies a dropout mask, which helps reducing the risk of overfitting by randomly shutting some nodes in the layer, on the extracted sequence.

In particular, the `VivitTubeletEmbeddings` class implements the patch extraction procedure using a 3D convolution that projects the input tensors onto an embedded dimension $\mathbb{R}^d$, which has been chosen according to the architecture of the original backbone, using a kernel of size (t, h, w) and introducing a stride of (t, h, w) among convolutions to ensure that the convolutional layer extracts non-overlapping patches of size (t, h, w).

Therefore, for an input batch of size (B, C, T, H, W), the `VivitEmbeddings` block creates a sequence containing $N = n_t \cdot n_h \cdot n_w$ tokens of dimension $\mathbb{R}^d$, with $n_t = \lfloor \frac{T}{t} \rfloor$, $n_h = \lfloor \frac{H}{h} \rfloor$ and $n_t = \lfloor \frac{W}{w} \rfloor$, for each tensor in the batch.

5.2 Implementing a single encoder block

The transformer encoder block features a sequence of 12 `VivitLayer` blocks, which are implemented by following the same structure as the original backbone.

In fact, each block starts by having the extracted sequence go through layer normalization before entering the attention block, which is followed by a second layer normalization before the sequence passes through the multilayer perceptron, after which a dropout mask is applied.

5.2.1 Implementing the attention mechanism

As mentioned at the beginning of *Chapter 3*, the introduction of the (self-)attention mechanism in the field of natural processing has driven researchers to adapt this mechanism for computer vision tasks as well.

The idea behind the attention mechanism lies in the fact that, at each step, the decoder can focus on specific parts of the input sequence depending on a context vector containing the representations of the input sequence at each time step.

For this reason, the attention mechanism makes use of three main types of representations: the query vector, which indicates on what elements a token should focus on, and a sequence of key-value pairs, where the key vectors represent the information provided by other tokens while the value vectors represent the actual content associated to these tokens [25].

Therefore, given a query vector $\mathbf{q}$ and a database $\mathcal{D} = \{(\mathbf{k}_1, \mathbf{v}_1), \ldots, (\mathbf{k}_m, \mathbf{v}_m)\}$

containing key-value pairs, the attention score is defined as

$$Attention(\mathbf{q}, \mathcal{D}) = \sum_{i=1}^{m} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i, \text{ with } \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i \in \mathbb{R}. \tag{5.1}$$

In particular, the attention scores $\alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i$ are often generated by some function $a(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i$ and normalized using a softmax function to ensure that the weights are non-negative and sum up to 1, meaning that

$$\alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i = \frac{e^{a(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i}}{\sum_j e^{a(\mathbf{q}, \mathbf{k}_j)\mathbf{v}_j}} \tag{5.2}$$

Concerning the structure of the `GaitViViT` model, the `VivitAttention` class implements the scaled dot-product attention mechanism, which is very similar to the spatio-temporal attention mechanism described in *Section 3.3.1* and corresponds to the default attention mechanism used by **Hugging Face**'s video processing models.

To ensure lower complexity while preserving correctness during computations, the `VivitAttention` class extracts queries, keys and values using separate linear layers and leaves computations to the `scaled_dot_product_attention` functional provided by **PyTorch**. After the scores are computed, the extracted features go through a linear layer and a dropout mask before entering the multilayer perceptron.

5.2.2 Implementing the multilayer perceptron

The `VivitMLP` class follows the same structure as the multilayer perceptron that was implemented for the original backbone, merely applying some optimization techniques based on the **Hugging Face** implementation.

In particular, the multilayer perceptron starts with a linear layer, which upscales the extracted features from the embedded dimension $\mathbb{R}^d$ to $\mathbb{R}^{4d}$, followed by a GELU non-linearity. To ensure that computations are optimized while maintaining accuracy, the activation function is implemented using the Fast GELU approximation [26], which can be described using the following equation:

$$GELU(x) = x\Phi(x) \approx 0.5x \cdot \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}} \cdot (x + 0.044715x^3)\right)\right) \tag{5.3}$$

Next, a first dropout mask is applied before the extracted features pass through a second linear layer, which downscales the extracted features from $\mathbb{R}^{4d}$ back to $\mathbb{R}^d$, followed by another dropout mask.

5.3 Adapting the output layer

While the original *Video Vision Transformer* was meant for multiclass video classification, the `GaitViViT` model needs to perform binary classification on the dataset.

For this reason, the `AnomalyHead` class features a linear layer, which upscales the classification token's extracted features to a higher-dimensional space, followed by a ReLU activation function and a second linear layer, which instead projects the features onto a one-dimensional logit representing the output of the network.

```
class AnomalyHead(nn.Module):
  def __init__(self, in_features, hidden_features):
    super().__init__()

    self.fc1 = nn.Linear(in_features, hidden_features)
    self.act = nn.ReLU()
    self.fc2 = nn.Linear(hidden_features, 1)

  def forward(self, x):
    x = self.fc1(x)
    x = self.act(x)
    logit = self.fc2(x)

    return logit
```

Listing 5.3. Structure of the `AnomalyHead` class.

This approach has been chosen because, due to the class imbalance of the dataset, the model will take an anomaly detection approach, allowing to use the `nn.BCEWithLogitsLoss` loss function during training.

5.4 Initializing the model with pre-trained weights

Along with the configuration of the original backbone, **Hugging Face** also provides a dictionary containing the weights of the original model, which can be loaded into the `GaitViViT` model to make the training procedure faster.

However, loading this dictionary as it is into the new model is not sufficient as the same components are memorised using different names between the models, resulting in a mismatch highlighted by the presence of missing keys, which represent the components in the `GaitViViT` model for which no corresponding weight in the original model could be found, and unexpected keys, which represent the components of the original model that might not be present in the `GaitViViT` model.

For this reason, the weights of the original model are loaded using a mapping function, which maps the weights of the `VivitForVideoClassification` class onto the structure of the `GaitViViT` model.

It should be noticed that, even though the backbone weights are correctly mapped onto the `GaitViViT` model, there appears to be a slight discrepancy in the encoder block. Since these differences tend to be in the order of 10^{-3} when using `float32` arithmetics and in the order of 10^{-8} when using `float64` arithmetics, the error is likely caused by hardware limitations and by the non-associative nature of floating-point operations, especially when working on large data.

Chapter 6

Training

6.1 Transfer learning

Transfer learning is a machine learning technique that involves using an existing model, which has been previously trained on large datasets for a specific task, to perform a new, but typically similar, task on a different dataset [27]. This strategy comes in handy to save time during training while mitigating the risk of overfitting as the dataset used for the new task tends to be too small for training the model from scratch.

Transfer learning strategies normally consist of freezing the original model's backbone and just training or modifying the final layers to adapt the model to the new task.

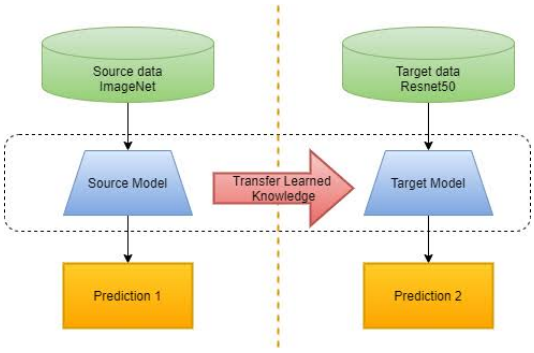


Figure 6.1. Example of a transfer learning pipeline for computer vision.

For this reason, the `GaitViViT` model will be trained using a hybrid approach that combines linear probing and full fine-tuning.

6.2 Constructing the training, validation and testing datasets

Creating the training, validation and testing datasets by simply splitting the unified dataset described in *Section 4.3.4* introduces two main issues to address: the class imbalance between healthy subjects and patients afflicted with Parkinson's disease,

as well as the risk of data leakage caused by the fact that multiple videos in the dataset might represent the same subject.

For this reason, the training, validation and testing datasets are constructed using **Scikit-learn**'s `StratifiedKFold` function [28], which combinaes K-fold cross-validation and stratified sampling to ensure that, for each fold, the class distribution is (approximately) preserved within all three subsets. A first strategy made use of the `StratifiedGroupKFold` function, which automatically prevents data leakage by allowing to split data with respect to a chosen group, although this version was ultimately discarded because the `StratifiedGroupKFold` function tends to be less consistent when preserving proportions across different folds.

Therefore, to ensure that data is split without causing leakage between the training, validation and testing datasets, the chosen approach consists of introducing a patient ID, which uniquely identifies the subjects in the dataset, and using it with the class labels to determine the splits. In particular, the first split is performed to isolate the subjects that will compose the testing dataset, which corresponds to 20% of the original dataset, and, for each generated fold, a second split is performed on the remaining data to determine the subjects that will compose the training and validation datasets, which correspond to 70% and 10% of the original dataset, respectively. The resulting information is then stored in a dictionary in order to keep track of the structure of the training, validation and testing datasets for each fold.

Split	Desired Proportion	Number of Items
Training	70%	≈ 744
Validation	10%	≈ 106
Testing	20%	≈ 212

Table 6.1. General idea of the training, validation and testing datasets.

This strategy ensures that, for each fold, the subsets have no common data or patients, although the desired proportions indicated in *Table 6.1* may not always be achieved. This result can be attributed to the behaviour of the `StratifiedKFold` function, which prioritizes maintaining the class distribution across the subsets over maintaining subset proportions, and the need to focus on group splitting in order to avoid data contamination.

6.3 Learning configurations

A hyperparameter is a configuration variable that is set before training in order to control the model's learning progress. For the purpose of this project, the hyperparameters will be chosen based on the configurations presented in Arnab et al.'s original model [12, Appendix A.2].

6.3.1 Gradient optimization

The model is optimized using stochastic gradient descent with momentum, a technique that updates model parameters using a running average between the previous

update and the current gradient, resulting in

$$\theta_{t+1} = \theta_t - \eta v_t \tag{6.1}$$
$$v_t = \beta v_{t-1} + \nabla \mathcal{L}(\theta_t) \tag{6.2}$$

Additionally, the fine-tuning phase introduces weight decay regularization, which is commonly used to improve model generalization and prevent overfitting, meaning that *Equation 6.2* becomes

$$\theta_{t+1} = \theta_t - \eta v_t \tag{6.3}$$
$$v_t = \beta v_{t-1} + \nabla \mathcal{L}(\theta_t) + \lambda \theta_t \tag{6.4}$$

Overall, this technique comes in handy to speed up training while maintaining gradient stability, although this algorithm may not always converge to the absolute minimum of the loss function and, as such, to the best parameters for the task.

6.3.2 Learning rate scheduler

During training, the model's learning rate is adjusted using the cosine annealing scheduling algorithm [29], which gradually reduces the starting learning rate $\eta_{\max}$ up to a minimum learning rate $\eta_{\min}$ according to the following equation:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{T_{\text{curr}}\pi}{T_{\max}}\right)\right) \tag{6.5}$$

The main advantage of the cosine annealing scheduling algorithm lies in its ability to ensure faster convergence while maintaining a smoother learning rate decay during training.

6.3.3 Loss function

Since the `GaitViViT` model has to perform a supervised anomaly detection task, its performance will be measured using **PyTorch**'s `nn.BCEWithLogitsLoss` function, which combines binary cross entropy loss with a sigmoid activation function. This means that the loss for a single input x_i and its logit y_i will be computed using the following equation:

$$\mathcal{L}_i = -w_i[p_i \cdot y_i \log(\sigma(x_i)) + (1 - y_i)\log(1 - \sigma(x_i))] \tag{6.6}$$
$$\sigma(x_i) = \frac{1}{1 + e^{-x_i}} \tag{6.7}$$

Generally speaking, this loss function can be defining by specifying optional arguments such as the individual weight w_i to assign to each element x_i and whether to perform a reduction, such as summing or averaging individual losses, as well as a "positive weight" p_i for the samples with label $y_i = 1$. However, for the purpose of this task, the loss function will consider just the last parameter, using the ratio between healthy subjects ($y_i = 0$) and subjects with Parkinson's disease ($y_i = 1$) as the positive weight. This means that *Equation 6.6* gets simplified to

$$\mathcal{L}_i = -[p_i \cdot y_i \log(\sigma(x_i)) + (1 - y_i)\log(1 - \sigma(x_i))] \tag{6.8}$$

6.4 Implementing the training algorithm

The main training algorithm is implemented as a three-step pipeline, starting with a short linear probing phase on the classification head followed by a full fine-tuning on the entire model and ending with a testing round on the best-performing model. For each of these steps, the resulting performance metrics are saved for further analysis and ablations.

Concerning the linear probing and fine-tuning steps, each epoch actually consists of a training phase and a validation phase.

During the training phase, the model is set in training mode so that it can run both the forward pass, which focuses on making predictions on the training data and computing the resulting error, and the backward pass, which uses the results of the forward pass to optimize the model and adjust the learning rate.

On the other hand, during the validation phase, the model is set in evaluation mode, which means that any stochastic component, such as dropout masks, is made deterministic and no backward pass will be run. Additionally, for each epoch, the validation F1-score is used to determine which model yields the best overall performance in order to save its weights for future steps and analysis. In particular, the F1-score is chosen because it tends to provide a more reliable performance estimation for unbalanced datasets compared to other commonly used metrics, such as accuracy.

6.4.1 Running linear probing on the classification head

Linear probing is a machine learning technique that consists of freezing a pre-trained backbone and training the final classification head for making predictions.

More specifically, the classification head is trained for 5 epochs using stochastic gradient descent with momentum and cosine annealing scheduling. Since this step only focuses on the final layers of the model, the starting learning rate is set to $\eta_{\text{start}} = \eta_{\max} = 3 \cdot 10^{-4}$ and no weight decay is applied. In particular, to ensure gradient stability and reduce the risk of overfitting during the fine-tuning phase, the learning rate is adjusted using **Hugging Face**'s `get_cosine_schedule_with_warmup` function, which introduces linear warmups during the first training epochs, resulting in the following equation:

$$\eta_t = \begin{cases} \eta_{\max} \cdot \frac{t}{T_{\text{warmup}}} & \text{if } t < T_{\text{warmup}} \\ \eta_{\max} \cdot \frac{1}{2}\left(1 + \cos\left(c\pi \cdot \frac{t - T_{\text{warmup}}}{T - T_{\text{warmup}}}\right)\right) & \text{if } t \geq T_{\text{warmup}} \end{cases} \tag{6.9}$$

In addition, whenever the validation F1-score registered during an epoch exceeds the current global maximum, the latter is updated and the weights of the current model are saved as a checkpoint for the fine-tuning phase.

6.4.2 Fine-tuning the entire model

Fine-tuning is a machine learning technique that aims to adapt a pre-trained model to a new task by further training it on a new, smaller dataset.

In particular, the entire model is fine-tuned using grid search to assess which combination of starting learning rate and weight decay yields the best performance.

Hyperparameter	Possible Values
Starting Learning Rate	$\{10^{-6}, 10^{-5}, 10^{-4}\}$
Weight Decay	$\{10^{-2}, 10^{-1}\}$

Table 6.2. Grid search parameters used during fine-tuning

For each hyperparameter combination described in *Table 6.2*, the model obtained at the end of the linear probing step is trained for 15 epochs, again using stochastic gradient descent with momentum and weight decay and cosine annealing scheduling.

In addition, whenever the validation F1-score registered during an epoch exceeds the current global maximum, the latter is updated and the weights of the current model are saved as a checkpoint for the testing phase, regardless of the hyperparameter combination.

However, since grid search tends to be computationally expensive and fine-tuning can result in overfitting, this phase makes use of an early stopping mechanism so that, whenever model performance on a certain hyperparameter combination does not appear to improve significantly, the corresponding training loop is terminated and the model is reset in order to work with another hyperparameter combination.

6.4.3 Testing the best model

After the model yielding the best results during the fine-tuning phase is saved, the testing phase aims to assess its learning abilities when dealing with unseen data.

For this reason, the testing phase simply consists of an evaluation phase on the testing dataset.

Chapter 7

Results

This chapter describes the results obtained after training the model according to the modalities described in *Chapter 6*.

For a more rigorous analysis, the model is trained using all the folds generated during the data split procedure described in *Section 6.2* and, to ensure more robust results, model performance is evaluated by reporting mean and standard deviation of each performance metric. Additionally, six versions of the model have been trained, with the differences being the upscaling factor and the activation function used in the final classification head, as described in *Table 7.1*.

Version	**Upscaling Factor (`cls_ratio`)**	**Activation Function (`self.act`)**
1	2.0	`nn.ReLU`
2	1.0	`nn.ReLU`
3	4.0	`nn.ReLU`
4	2.0	`nn.GELU`
5	1.0	`nn.GELU`
6	4.0	`nn.GELU`

Table 7.1. Configurations of the classification head for each trained version.

To avoid computational and memory overhead, each version of the model is trained on a NVIDIA A100 GPU [30] using gradient accumulation and mixed precision arithmetic.

7.1 Performance evaluation metrics

The performance of a (multi-)classification model is often measured in terms of its accuracy, which denotes the proportion of correct predictions $\mathcal{P} = \{\hat{y}_i\}_{i=1}^{N}$ made on a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$.

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}(\hat{y}_i = y_i), \text{ where } \mathbf{1}(\hat{y}_i = y_i) = \begin{cases} 1 & \text{if } \hat{y}_i = y_i \\ 0 & \text{otherwise} \end{cases} \tag{7.1}$$

However, when working with imbalanced datasets, accuracy tends to be a misleading metric because a model might be able to achieve high accuracy performance

by making correct predictions on the majority while struggling on the more relevant minority class.

For this reason, whenever a binary classification model is trained on an imbalanced dataset, it is better practice to assess its performance using the F1-score metric, which is computed as the harmonic mean between the model's precision, which represents the proportion of correct predictions among all positive predictions, and the model's recall, which represents the proportion of correct positive predictions among the ground-truth positive labels.

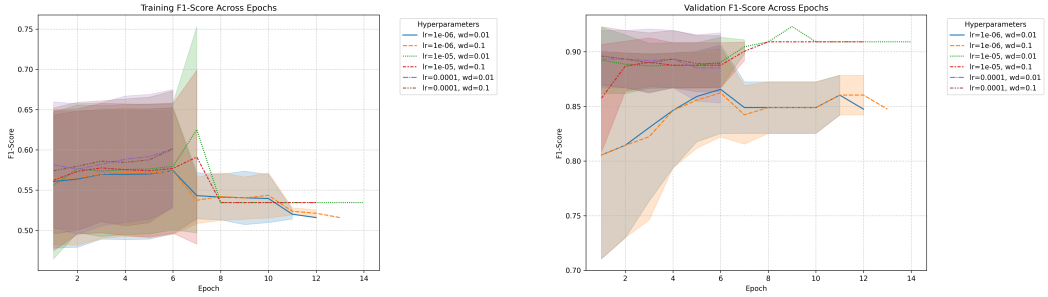
$$\mathrm{F1} = 2 \cdot \frac{\mathrm{Precision} \cdot \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}} \tag{7.2}$$

$$\mathrm{Precision} = \frac{\mathrm{True\ Positives}}{\mathrm{True\ Positives} + \mathrm{False\ Positives}} \tag{7.3}$$

$$\mathrm{Recall} = \frac{\mathrm{True\ Positives}}{\mathrm{True\ Positives} + \mathrm{False\ Negatives}} \tag{7.4}$$

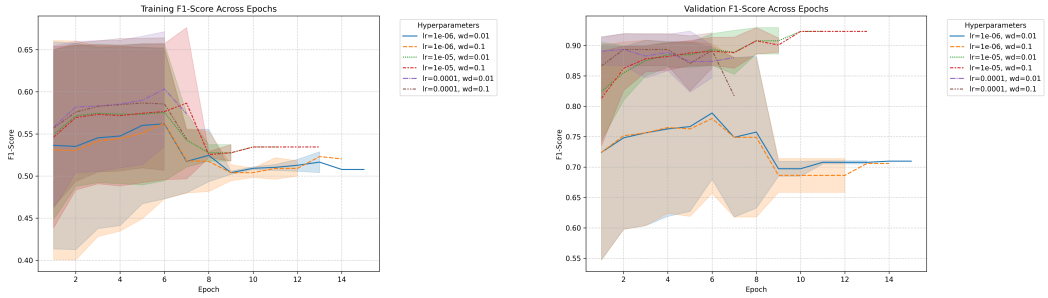
The main advantage of assessing a classification model using its F1-score lies in the fact that the model will be considered to achieve high performance only if both its precision and recall are high, thus providing a more reliable performance assessment when working with imbalanced datasets.

7.2 Performance analysis



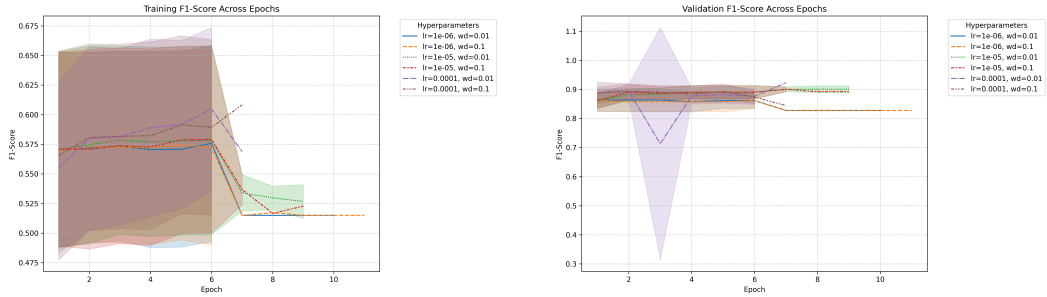
(a) Training F1-score across epochs. **(b)** Validation F1-score across epochs.

Figure 7.1. Overall F1-score for each hyperparameter combination - Version 1.



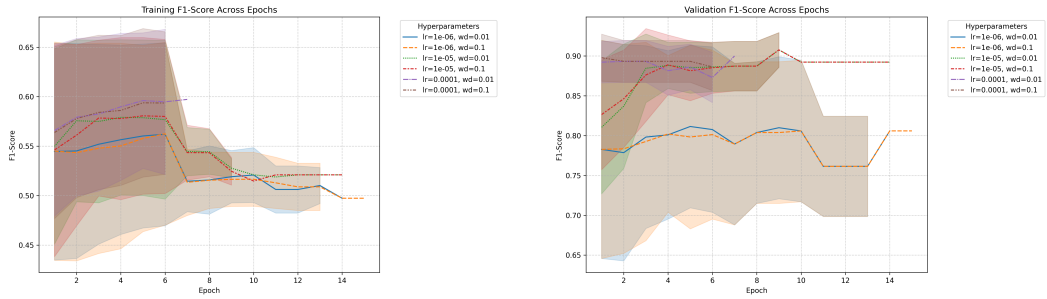
(a) Training F1-score across epochs. **(b)** Validation F1-score across epochs.

Figure 7.2. Overall F1-score for each hyperparameter combination - Version 2.



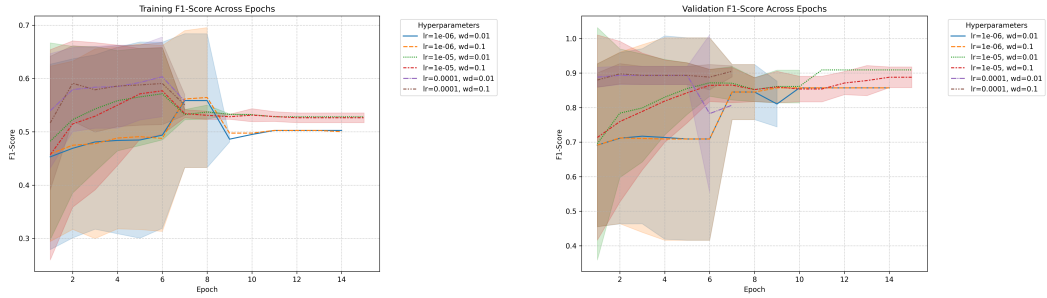
(a) Training F1-score across epochs. **(b)** Validation F1-score across epochs.

Figure 7.3. Overall F1-score for each hyperparameter combination - Version 3.



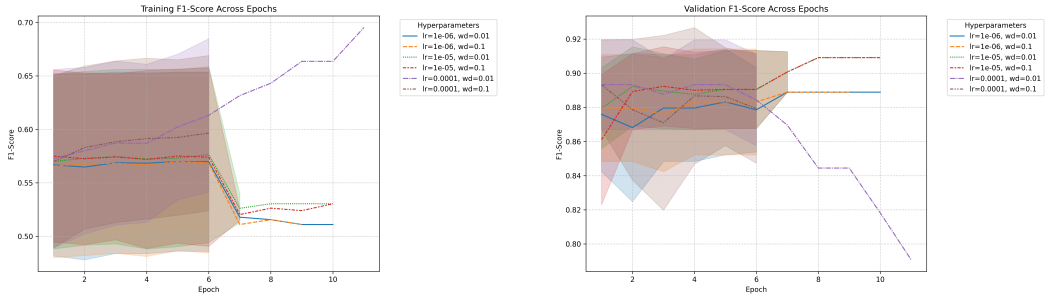
(a) Training F1-score across epochs. **(b)** Validation F1-score across epochs.

Figure 7.4. Overall F1-score for each hyperparameter combination - Version 4.



(a) Training F1-score across epochs. **(b)** Validation F1-score across epochs.

Figure 7.5. Overall F1-score for each hyperparameter combination - Version 5.



(a) Training F1-score across epochs. **(b)** Validation F1-score across epochs.

Figure 7.6. Overall F1-score for each hyperparameter combination - Version 6.

Version	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Overall (mean ± std)
1	0.567	0.667	0.698	0.403	0.815	0.630 ± 0.155
2	0.697	0.654	0.677	0.393	0.800	0.644 ± 0.151
3	0.754	0.680	0.633	0.403	0.800	0.654 ± 0.154
4	0.794	0.667	0.656	0.390	0.800	0.661 ± 0.166
5	0.667	0.680	0.712	0.421	0.821	0.660 ± 0.147
6	0.783	0.680	0.655	0.400	0.800	0.664 ± 0.160

Table 7.2. Testing F1-score across folds for each version described in *Table 7.1*.

Version	Precision (mean ± std)	Recall (mean ± std)	F1-Score (mean ± std)
1	0.509 ± 0.164	0.909 ± 0.181	0.630 ± 0.155
2	0.513 ± 0.169	0.933 ± 0.080	0.644 ± 0.151
3	0.521 ± 0.171	0.943 ± 0.061	0.654 ± 0.154
4	0.527 ± 0.183	0.951 ± 0.034	0.661 ± 0.166
5	0.538 ± 0.167	0.928 ± 0.135	0.660 ± 0.147
6	0.531 ± 0.176	0.950 ± 0.057	0.664 ± 0.160

Table 7.3. Testing performance for each version described in *Table 7.1*.

A comparison between the training and validation performance shown in *Figures 7.1 to 7.6* and the testing performance reported in *Table 7.2* reveals that each trained version struggles with generalizing to unseen testing data, achieving lower and fluctuating overall F1-scores. More specifically, *Table 7.3* shows that this performance degradation is mainly caused by each version's low and fluctuating precision, whereas the recall stays consistently high regardless of the classification head used by the model.

These results can be explained by considering that a small dataset of just approximately 1000 videos is not sufficient to properly train a complex transformer-based architecture. In particular, splitting an imbalanced dataset into stratified subsets can often result in the creation of unrepresentative testing datasets containing a very small number of videos belonging to the minority class, limitin the model's learning and generalization capabilities.

7.2.1 Testing alternative models

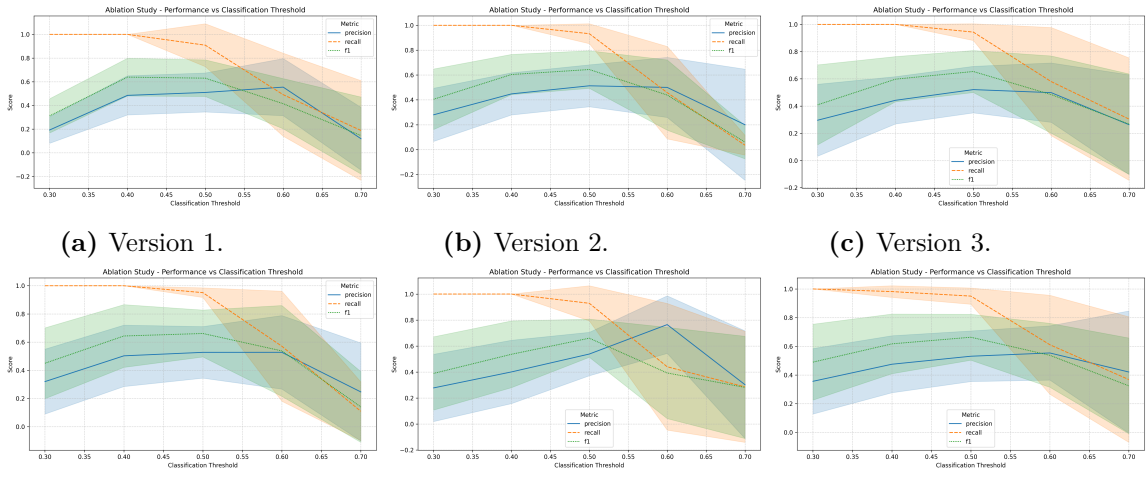
In order to better investigate the results achieved by the `GaitViViT` model, the tensor dataset was used to train some different architectures on the same task.

In particular, the first chosen architecture was a *cascaded CNN-LSTM model* [31] that consisted of a pre-trained *ResNet* backbone [32], which was used for convolution-based feature extraction, and a *long short-term memory network* [33], which focused on modelling spatio-temporal attention between extracted features, and ended with the `GaitViViT` classification head described in *Section 5.3*. Training this architecture under the same settings described in *Chapter 6* yields performance similar to the `GaitViViT` model, but with an overall higher loss, suggesting that a cascaded model would be even worse for the proposed task. This result can be explained by the fact that cascaded models typically struggle to generalize on imbalanced datasets, as opposed to transformer-based architectures.

The second chosen architecture was a *few-shot action detection model* [34] that combined the `GaitViViT` backbone with a temporal cross transformer, which is used to model attention between different temporal frames. Training this architecture using episodic learning, which consists of making predictions on unseen query data based on known support data, yields consistently high performance, with the model achieving a testing F1-score of 0.922 ± 0.051, suggesting that few-shot models can be more convenient than transformers when working on smaller, imbalanced datasets.

7.3 Threshold ablation study

Due to the medical purpose of this project, a crucial research aspect lies in understanding which classification threshold would be more suitable based on the task that needs to be performed [35].



(a) Version 1. **(b)** Version 2. **(c)** Version 3.

(d) Version 4. **(e)** Version 5. **(f)** Version 6.

Figure 7.7. Threshold ablation study for each version of the `Gait-ViViT` model.

In fact, a lower classification threshold would make the model more sensitive towards positive examples, making it more likely to detect patients showing early

or mild symptoms but also more vulnerable to false positives, which could result in requiring additional screenings before patients start undergoing more invasive or expensive treatments. By contrast, a higher classification threshold would require the model to be more confident when flagging a positive examples, making it less likely to predict false positives but more vulnerable to false negatives, which means that the model might overlook patients showing little or subtle symptoms.

Overall, the plots shown in *Figure* 7.7 corroborate the hypothesis that the `Gait-ViViT` model struggles to generalize on unseen data, as shown by the fluctuating performance achieved across different classification thresholds. More specifically, setting a lower classification threshold causes the model to achieve lower precision, whereas the recall tends to stay high and show little to no fluctuation, suggesting that most model errors under these settings would be represented by false positives. By contrast, setting a higher classification threshold results in further performance degradation as, despite a slight increase in precision, the recall starts decreasing and showing a lot of fluctuation, resulting in a lower overall F1-score and in the model becoming less reliable as it becomes more vulnerable to false negatives as well.

7.3.1 Extending the ablation study to the few-shot model

Since the *few-shot action detection model* described in *Section* 7.2.1 appeared to achieve consistently good performance, the ablation study was extended to determine whether this architecture could be suited for a large-scale application

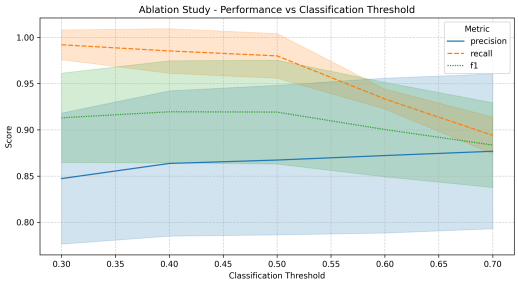


Figure 7.8. Threshold ablation study on the few-shot version of the model.

The results shown in *Figure* 7.8 suggest that the *few-shot action detection* model can represent a reliable alternative to the transformer-based architecture as it is able to achieve consistently good results using any of the indicated classification thresholds. In particular, setting a lower classification threshold causes the model to achieve very high recall and a slightly lower precision, both with smaller fluctuation compared to the transformer-based model, making the model reliable but still vulnerable to false positives. By contrast, setting a higher classification threshold allows the model to improve its precision, thus reducing the risk of false positives, but at the cost of a lower recall, introducing the risk of false negatives instead.

Chapter 8

Conclusions

This report introduces a transformer-based video processing model that aims to determine whether a subject has Parkinson's disease or not by simply looking at their overall gait.

The project pipeline starts by rendering data into synthetic skeleton videos and uniformly extracting frames from each video in order to capture the subject's main gait features. These frames are then used to train the model, whose backbone closely follows the structure of the *Video Vision Transformer* architecture, although the classification head is modified in order to make the model suitable for supervised anomaly detection.

This model is trained using a hybrid transfer learning approach that makes use of linear probing, which is used to train the final classification head, and full fine-tuning, which focuses on adapting the pre-trained backbone to the new task. For a more robust analysis, the model is trained using different versions of the classification head, which differ by the upscaling factor and/or by the activation function, using 5-fold cross-validation to obtain more reliable performance measurements. Additionally, an ablation study is performed on the best-performing models in order to determine their usability as the classification threshold changes.

Performance evaluation reveals that this transformer-based model struggles to generalize on unseen data, with the ablation study suggesting that performance tends to degrade as the classification increases due to the model's vulnerability to false positives, although these results can be attributed to the fact that the model has been trained on a small and imbalanced dataset, which is insufficient for a transformer-based architecture. In fact, a quick comparison with a simpler few-shot model reveals that, even when trained on a small dataset, the latter is able to better generalized, as corroborated by the results obtain during the ablation study for this architecture.

8.1 Strengths and current limitations

The novelty of the model lies in the fact that, rather than focusing on more specific symptoms of Parkinson's disease, such as bradykinesia, it focuses on gait abnormality as a whole. Additionally, training the model using a transfer learning strategy, instead of training it from scratch, makes it less computationally expensive and more viable

to large-scale medical usage. Furthermore, using synthetic videos featuring just the subject's extracted skeleton on a black background helps complying with privacy regulations raised by data protection law, such as the General Data Protection Law [36] [37] or the California Privacy Rights Act [38].

However, the model still shows several limitations concerning performance and portability. For instance, the lack of larger and uniform datasets requires merging and adapting different data that might contain different information, making the model harder to train or less reliable. Additionally, training the model using transfer learning requires implementing an architecture that remains as close as possible to the default *Video Vision Transformer* because, when mapping the backbone's original weights onto a different architecture, the resulting error becomes too large to ensure a proper training.

8.2 Future work

Overall, future work could focus on making the current architecture more flexible. For instance, one could focus on adapting the model's backbone to match other existing video processing models, such as the alternative versions of the *Video Vision Transformer* architecture or the *TimeSformer* model, or by investigating whether sampling a different number of frames from each video could influence the model's overall performance.

Another research question could focus on whether training the model on videos showing the patient and their surroundings can help it achieve better performance compared to training it on synthetic videos, although this research would require publicly available video datasets or, at least, the subjects' consensus for the study.

Bibliography

[1] MHS Tosin et al. "In-Home Remote Assessment of the MDS-UPDRS Part III: Multi-Cultural Development and Validation of a Guide for Patients." In: *Movement disorders clinical practice* 11.12 (2024), pp. 1576–1581. DOI: 10.1002/mdc3.14203. URL: https://pubmed.ncbi.nlm.nih.gov/39310988/.

[2] KG Sibley et al. "Video-Based Analyses of Parkinson's Disease Severity: A Brief Review." In: *Journal of Parkinson's disease* 11.s1 (2021), S83–S93. DOI: 10.3233/JPD-202402. URL: https://pubmed.ncbi.nlm.nih.gov/33682727/.

[3] AP Dennis and AP Strafella. "The role of AI and machine learning in the diagnosis of Parkinson's disease and atypical parkinsonisms." In: *Parkinsonism related disorders* 126 (2024), p. 106986. DOI: 10.1016/j.parkreldis.2024.106986. URL: https://pubmed.ncbi.nlm.nih.gov/38724317/.

[4] DL Guarín et al. "Automated video analysis for early detection of bradykinesia in Parkinson's disease." In: *Journal of neuroengineering and rehabilitation* 23.1 (2026). DOI: 10.1186/s12984-026-01885-z. URL: https://pubmed.ncbi.nlm.nih.gov/41664165/.

[5] Tee Connie et al. "Pose-Based Gait Analysis for Diagnosis of Parkinson's Disease". In: *Algorithms* 15.12 (2022). ISSN: 1999-4893. DOI: 10.3390/a15120474. URL: https://www.mdpi.com/1999-4893/15/12/474.

[6] L. Musso et al. "F-BioGate: A Multimodal Dataset for Frontal Video-Based Gait Biometric Recognition". In: *Proceedings of the IEEE/IAPR International Joint Conference on Biometrics (IJCB)*. In press. 2026.

[7] Keiron O'Shea and Ryan Nash. *An Introduction to Convolutional Neural Networks*. 2015. arXiv: 1511.08458 [cs.NE]. URL: https://arxiv.org/abs/1511.08458.

[8] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

[9] Xuran Pan et al. *On the Integration of Self-Attention and Convolution*. 2022. arXiv: 2111.14556 [cs.CV]. URL: https://arxiv.org/abs/2111.14556.

[10] Alexey Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV]. URL: https://arxiv.org/abs/2010.11929.

[11] Ze Liu et al. *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*. 2021. arXiv: 2103.14030 [cs.CV]. URL: https://arxiv.org/abs/2103.14030.

[12] Anurag Arnab et al. *ViViT: A Video Vision Transformer*. 2021. arXiv: 2103.15691 [cs.CV]. URL: https://arxiv.org/abs/2103.15691.

[13] Gedas Bertasius, Heng Wang, and Lorenzo Torresani. *Is Space-Time Attention All You Need for Video Understanding?* 2021. arXiv: 2102.05095 [cs.CV]. URL: https://arxiv.org/abs/2102.05095.

[14] Hao-Shu Fang et al. "AlphaPose: Whole-Body Regional Multi-Person Pose Estimation and Tracking in Real-Time". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2022).

[15] G. Bradski. "The OpenCV Library". In: *Dr. Dobb's Journal of Software Tools* (2000).

[16] Charles R. Harris et al. "Array programming with NumPy". In: *Nature* 585.7825 (Sept. 2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2. URL: https://doi.org/10.1038/s41586-020-2649-2.

[17] Almar Klein et al. *imageio/imageio: v2.37.4*. Version v2.37.4. July 2026. DOI: 10.5281/zenodo.21449723. URL: https://doi.org/10.5281/zenodo.21449723.

[18] Filipe Gama et al. "Automatic infant 2D pose estimation from videos: Comparing seven deep neural network methods". In: *Behavior Research Methods* 57.10 (2025). ISSN: 1554-3528. DOI: 10.3758/s13428-025-02816-x. URL: http://dx.doi.org/10.3758/s13428-025-02816-x.

[19] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. arXiv: 1912.01703 [cs.LG]. URL: https://arxiv.org/abs/1912.01703.

[20] TorchVision maintainers and contributors. *TorchVision: PyTorch's Computer Vision library*. https://github.com/pytorch/vision. 2016.

[21] Marcel Simon, Erik Rodner, and Joachim Denzler. *ImageNet pre-trained models with batch normalization*. 2016. arXiv: 1612.01452 [cs.CV]. URL: https://arxiv.org/abs/1612.01452.

[22] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Ed. by F. Pereira et al. Vol. 25. Curran Associates, Inc., 2012. URL: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf.

[23] Thomas Wolf et al. "Transformers: State-of-the-Art Natural Language Processing". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45. URL: https://aclanthology.org/2020.emnlp-demos.6/.

[24] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. *Layer Normalization*. 2016. arXiv: 1607.06450 [stat.ML]. URL: https://arxiv.org/abs/1607.06450.

[25] Aston Zhang et al. *Dive into Deep Learning*. https://D2L.ai. Cambridge University Press, 2023.

[26] Dan Hendrycks and Kevin Gimpel. *Gaussian Error Linear Units (GELUs)*. 2023. arXiv: 1606.08415 [cs.LG]. URL: https://arxiv.org/abs/1606.08415.

[27] Fuzhen Zhuang et al. *A Comprehensive Survey on Transfer Learning*. 2020. arXiv: 1911.02685 [cs.LG]. URL: https://arxiv.org/abs/1911.02685.

[28] F. Pedregosa et al. "Scikit-learn: Machine Learning in Python". In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.

[29] Ilya Loshchilov and Frank Hutter. *SGDR: Stochastic Gradient Descent with Warm Restarts*. 2017. arXiv: 1608.03983 [cs.LG]. URL: https://arxiv.org/abs/1608.03983.

[30] NVIDIA Corporation. *NVIDIA A100 Tensor Core GPU*. https://www.nvidia.com/en-us/data-center/a100/. Accessed: 2026-09-11. 2020.

[31] Jeff Donahue et al. *Long-term Recurrent Convolutional Networks for Visual Recognition and Description*. 2016. arXiv: 1411.4389 [cs.CV]. URL: https://arxiv.org/abs/1411.4389.

[32] Kaiming He et al. *Deep Residual Learning for Image Recognition*. 2015. arXiv: 1512.03385 [cs.CV]. URL: https://arxiv.org/abs/1512.03385.

[33] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735. URL: https://dl.acm.org/doi/10.1162/neco.1997.9.8.1735.

[34] "Temporal Relational CrossTransformers for Few-Shot Action Recognition". In: *Computer Vision and Pattern Recognition*. 2021.

[35] S. A. Hicks et al. "On evaluation metrics for medical applications of artificial intelligence". In: *Scientific Reports* 12.1 (2022), p. 5993. DOI: 10.1038/s41598-022-09954-8. URL: https://www.nature.com/articles/s41598-022-09954-8.

[36] European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation)*. 2016. URL: https://eur-lex.europa.eu/eli/reg/2016/679/oj.

[37] Kalle Hjerppe, Jukka Ruohonen, and Ville Leppanen. "The General Data Protection Regulation: Requirements, Architectures, and Constraints". In: *2019 IEEE 27th International Requirements Engineering Conference (RE)*. IEEE, 2019, pp. 265–275. DOI: 10.1109/re.2019.00036. URL: http://dx.doi.org/10.1109/RE.2019.00036.

[38] State of California. *California Privacy Rights Act of 2020 (CPRA)*. Cal. Civ. Code § 1798.100 et seq. Approved by voters as Proposition 24 on Nov. 3, 2020; operative Jan. 1, 2023. 2020. URL: https://oag.ca.gov/privacy/ccpa.